

BOOK 2

DragonHub

Architecture Specification

Technical Architecture & Engineering Handbook

Version 1.0

Marko Turk

OpenAI ChatGPT

Internal Engineering Documentation

20 September 2026

Revision History

Version	Date	Changes
1.0	20 September 2026	Initial release - Architecture Specification

Table of Contents

Revision History	2
Part I - Core Architecture	4
Chapter 1 System Architecture & Core Principles	5
Chapter 2 Player Registry	12
Chapter 3 Club Registry	20
Chapter 4 Competition Registry	27
Chapter 5 Relationship Engine	35
Part II - Core Engines	42
Chapter 6 Statistics Engine	43
Chapter 7 Fixture Engine	51
Chapter 8 Synchronization & Background Jobs Engine	57
Chapter 9 Intelligence Engine	64
Chapter 10 Global Search Engine	71
Chapter 11 Notification & Workflow Engine	78
Chapter 12 Component Engine & UI Framework	84
Chapter 13 Event Bus & Workflow Engine	92
Chapter 14 Data Flow & Lifecycle	99
Chapter 15 Feature Development Framework	106
Chapter 16 Knowledge Management Engine	113
Chapter 17 Workflow Automation Engine	120
Part III - Infrastructure	126
Chapter 18 Integration Layer & External Data Providers	127
Chapter 19 Security, Permissions & Audit Architecture	133
Chapter 20 Scalability, Infrastructure & Deployment	140
Part IV - Platform Foundation	148
Chapter 21 Engineering Playbook	149
Chapter 22 Database Architecture & Schema Design	156
Chapter 23 API Architecture & Service Layer	165
Chapter 24 Quality Assurance, Testing & Release Management	171
Part V - Engineering Governance	178
Chapter 25 Decision Framework & Product Governance	179
Chapter 26 Engineering Constitution	186
Glossary	191
Abbreviations	194
Index	195

Part I - Core Architecture

Chapter 1 System Architecture & Core Principles

Chapter 2 Player Registry

Chapter 3 Club Registry

Chapter 4 Competition Registry

Chapter 5 Relationship Engine

[Architecture Diagram Placeholder]

Chapter 1

System Architecture & Core Principles

1.1 Purpose

The purpose of this document is to describe the technical architecture of DragonHub.

Unlike the Product Specification, which defines the product vision and workflows, this document defines:

- data architecture,
- relationships,
- system responsibilities,
- synchronization,
- registries,
- backend principles,
- engineering standards.

This document becomes the single source of truth for every future developer.

1.2 Engineering Philosophy

DragonHub is designed as a **Basketball Operating System**.

Every technical decision should support basketball workflows.

Engineering decisions should never be made independently from the product philosophy.

Whenever multiple technical solutions exist, choose the one that best supports basketball operations, even if it requires slightly more engineering work.

Architecture

1.3 Core Architecture

DragonHub is built around three permanent registries.

Figure 1.1 Core architecture



Every module consumes these registries.
No module owns independent player, club or competition objects.

1.4 Single Source of Truth

Every entity exists only once.

Player

One PlayerID.
Never duplicate players.

Club

One ClubID.
Every club references the same object.

Competition

One CompetitionID.
Every module references the same competition.
This principle is absolute.
No exceptions.

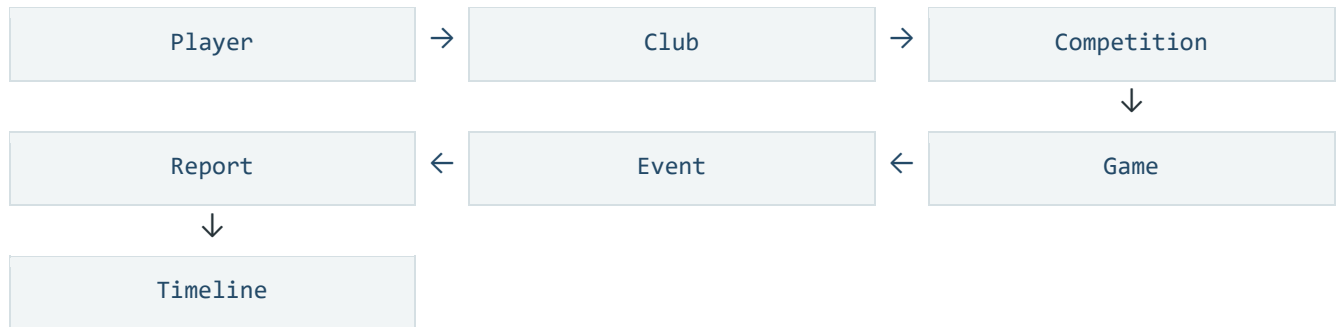
1.5 Relationship Philosophy

DragonHub is built as a connected knowledge graph.

Objects are connected rather than duplicated.

Example

Figure 1.1 Workflow



The user should never manually recreate relationships.

Everything should happen automatically.

1.6 Layered Architecture

DragonHub consists of five logical layers.

Layer 1

Public Data

Statistics

Fixtures

Standings

Rosters

Club Information

Player Information

Imported.

Never manually maintained.

Layer 2

Registries

Player Registry

Club Registry

Competition Registry

These create permanent identities.

Layer 3

Relationship Engine

Connects

Players

Clubs

Competitions

Games

Events

Reports

Everything references IDs.

Layer 4

Basketball Modules

Dashboard

Calendar

Events

Reports

Scouting Matchup

Player Database

Tournament Workspace

Watchlist

Review Queue

Shortlist

These consume information.

Never own information.

Layer 5

User Experience

Search

Filters

Drawers

Saved Views

Notifications

Theme

Permissions

Interaction layer only.

1.7 Module Ownership

Every module owns exactly one responsibility.

Example

Dashboard → Operations

Calendar → Club Operations

```
Events → Scouting Opportunities
Tournament Workspace → Tournament Scouting
Scouting Matchup → Live Scouting
Reports → Knowledge
Player Database → Discovery
Player Profile → Player Intelligence
Watchlist → Monitoring
Review Queue → Tasks
Shortlist → Recruitment
```

No module should duplicate another.

1.9 Event Flow

Every basketball activity follows the same flow.

```
Competition → Club → Roster → Players → Game → Observation → Report → Timeline → Watchlist
→ Review Queue → Shortlist
```

Every module fits into this architecture.

1.10 Product Separation

DragonHub separates:

```
Operational Planning → Dashboard
```

Calendar

```
Scouting → Events
```

Tournament Workspace

Scouting Matchup

```
Knowledge → Reports
```

Timeline

```
Recruitment → Watchlist
```

Review Queue

Shortlist

This separation should never be broken.

1.12 Long-term Architecture

Eventually there should no longer be:

```
DragonHub → Embedded Databases
```

Instead

DragonHub becomes one application.

Every module shares

Navigation

Theme

Registries

Permissions

Search

Timeline

Design System

Everything becomes native.

Rules

1.8 Architecture Rules

Every future feature must satisfy the following rules.

Rule 1

Never duplicate data.

Rule 2

Never compare names.

Always compare IDs.

Rule 3

Always reuse registries.

Rule 4

Always reuse existing components.

Rule 5

Always prefer relationships over copies.

Rule 6

Always update timelines automatically.

Rule 7

Always support progressive loading.

1.11 Engineering Principles

Every developer should remember:

Do not ask

"Where should this code go?"

Instead ask

"Which existing system already owns this responsibility?"

If the answer exists,

reuse it.

If no answer exists,

only then create something new.

Approved Architecture Principles

- DragonHub is a Basketball Operating System.
- One Player Registry.
- One Club Registry.
- One Competition Registry.
- Everything references IDs.
- Modules consume data rather than own it.
- Relationships replace duplication.
- Public data supports internal knowledge.
- Progressive loading is mandatory.
- Architecture always follows basketball workflows.

1.13 Success Criteria

A developer joining the project should understand within one hour:

- how DragonHub is organized,
- where every feature belongs,
- which registry owns each object,
- how modules communicate,
- and where new functionality should be implemented.

The architecture should make future development easier rather than more complex.

Architecture Notes

I actually think this document will become even more valuable than the Product Specification for Claude and Codex. The Product Specification tells them **what** DragonHub should be; the Architecture Specification tells them **where every piece belongs**. That distinction is exactly what has been missing when implementing larger features like Tournament Workspace, Club Registry, and the Scouting Matchup redesign.

Related chapters

[Chapter 2 - Player Registry](#)

[Chapter 3 - Club Registry](#)

[Chapter 4 - Competition Registry](#)

[Chapter 5 - Relationship Engine](#)

[Chapter 20 - Scalability, Infrastructure & Deployment](#)

[Chapter 26 - Engineering Constitution](#)

End of Chapter 1

Chapter 2

Player Registry

2.1 Purpose

The Player Registry is the foundation of DragonHub.

Every basketball player inside DragonHub exists exactly once.

Every module references the same PlayerID.

The Player Registry eliminates duplicate players and allows every observation, report and statistic to remain connected throughout the player's entire career.

2.2 Philosophy

A player is not a database row.

A player is a permanent basketball object.

Everything else connects to that object.

Examples

Reports → Timeline → Games → Statistics → Watchlist → Review Queue → Shortlist → Club History → National Team History → Media

Nothing should ever create another version of the same player.

Architecture

2.3 Permanent Identity

Every player receives

PlayerID

This ID never changes.

Not after:

Club transfer

League change

Competition

Promotion

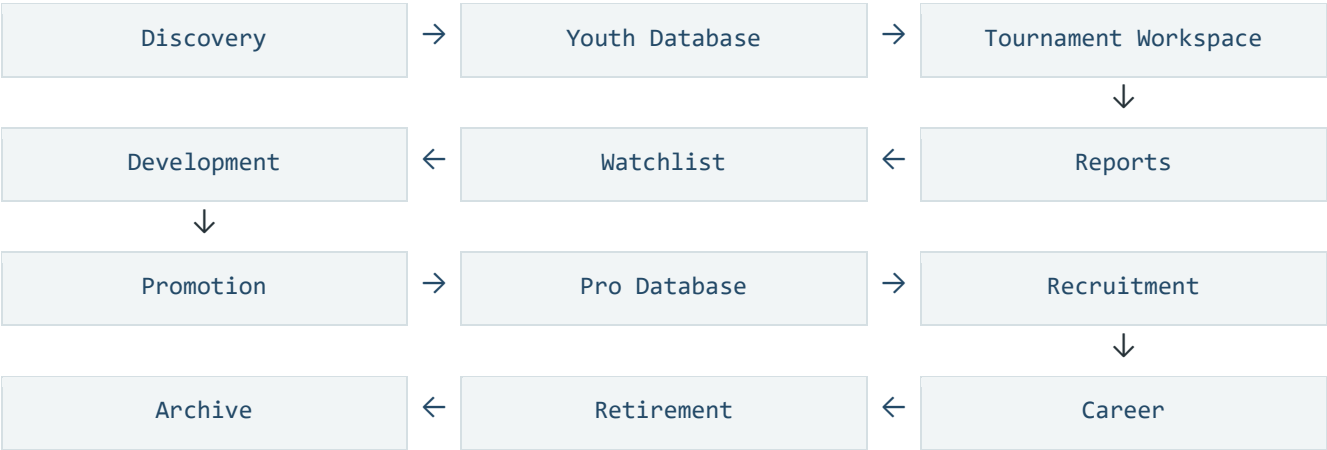
Retirement

Everything references PlayerID.

2.4 Player Lifecycle [Key principle]

Every player follows one lifecycle.

Figure 2.1 Workflow



PlayerID remains identical.

No migration.

No duplication.

2.5 Player Object

Every player contains

Identity

PlayerID

Full Name

Aliases

Preferred Name

Birth Date

Nationality

Passport(s)

Dominant Hand

Position(s)

Height

Weight

Wingspan (future)

Standing Reach (future)

Current Basketball

ClubID

CompetitionID

Team

League

Season

Jersey Number

Contract (optional)

Agent (optional)

Scouting

Estimated Level

Estimated Ceiling

Follow

Watchlist

Review Queue

Shortlist

Automatic Signals

Scout Summary

Latest Recommendation

Statistics

Current Season

Career

Game Logs

Advanced Metrics

Statistics Source

Last Updated

Refresh Schedule

Relationships

Reports

Games

Events

Tournaments

Videos

Timeline

Club History

National Team History

Everything references IDs.

2.6 Aliases

Players may have multiple names.

Examples

Aleksandar → Alexander

Misko → Miško

Diacritics → Normalized

DragonHub searches aliases automatically.

Only one PlayerID exists.

2.7 Duplicate Detection

Whenever importing players

DragonHub attempts

1.

PlayerID

2.

Official Identifier

3.

Name + Birth Date

4.

Alias

5.

Fuzzy Match

Only after all fail → Create New Player.

2.8 Promotion

One of DragonHub's most important workflows.

Youth Player → Professional Contract → Same PlayerID

Nothing moves.

Nothing is copied.

Player Category changes

Youth → Professional

Everything else remains.

2.9 Timeline

Every player automatically owns a Timeline.

Timeline displays

Game Notebook

Tournament Observation

Reports

Club Changes

Watchlist

Review Queue

Shortlist

Everything chronological.

Timeline never stores opinions.

Only events.

2.10 Scout Summary

Every player contains exactly one current Scout Summary.

The Scout Summary answers

"What do we currently think?"

Reports remain historical.

Scout Summary represents today's evaluation.

2.11 Automatic Signals

Signals belong to Player Registry.

Examples

Coming out of College

Rookie in Europe

Breakout

National Team Debut

High Upside

Signals are recalculated after every import.

Never manually entered.

2.12 Photos

Priority

Profile Photo → Fallback Initials

Never render

Photo

+

Initials

simultaneously.

Future imports automatically replace placeholders.

2.13 Statistics

Player Registry never owns statistics.

Statistics belong to Statistics Engine.

Player Registry stores only

Current Statistics Reference

Everything else comes from the Statistics Engine.

2.14 Club Changes

Changing clubs

does not modify PlayerID.

Instead

Club History

receives another entry.

Example

```
2024
Žalgiris
↓
2025
Baskonia
↓
2026
Olympiacos
```

Everything remains chronological.

2.15 Relationships

Player Registry never stores reports.

It stores references.

Example

```
Player → ReportID → Report
```

Exactly the same for

Games

Videos

Events

Tournaments

Timeline

Everything references IDs.

2.16 Search

Player Registry powers every search.

Searching

```
Karl Chavis → Player → Reports → Games → Timeline → Club → Competition
```

Everything immediately available.

2.17 Performance

Player Registry loads

Identity → Relationships → Profile → Statistics → Media

Never wait for statistics before rendering the player.

Rules

2.18 Must Never Change

One PlayerID.

Never duplicate players.

Promotion never creates another profile.

Everything references PlayerID.

Reports remain connected forever.

Timeline remains chronological.

Player Registry owns identity.

Not statistics.

Approved Player Registry Principles

- One PlayerID forever.
- No duplicate players.
- Promotion never creates a second profile.
- Reports, observations and timelines reference PlayerID.
- Scout Summary represents today's opinion.
- Reports preserve historical opinions.
- Statistics belong to the Statistics Engine.
- Profile photos always replace initials.
- Player Registry becomes the heart of DragonHub.

2.19 Success Criteria

Opening a player should immediately display

Everything DragonHub knows about him.

The user should never wonder

"Where did I write that report?"

Player Registry already knows.

Architecture Notes

The Player Registry is the **most important object in DragonHub**.

Every module ultimately exists to enrich the Player Registry:

- Tournament Workspace discovers players.
- Game Notebook evaluates players.
- Reports preserve scouting knowledge.
- Watchlist monitors players.
- Review Queue organizes work.
- Shortlist supports recruitment.

Everything eventually returns to the Player Registry.

If the Player Registry is designed correctly, every future feature becomes easier because it always has a single, permanent object to build upon. It should therefore be considered the most important architectural component in the entire DragonHub ecosystem.

Related chapters

[Chapter 3 - Club Registry](#)

[Chapter 4 - Competition Registry](#)

[Chapter 5 - Relationship Engine](#)

[Chapter 6 - Statistics Engine](#)

[Chapter 14 - Data Flow & Lifecycle](#)

[Chapter 22 - Database Architecture & Schema Design](#)

End of Chapter 2

Chapter 3

Club Registry

3.1 Purpose

The Club Registry is DragonHub's permanent registry of every basketball organization.

Every basketball organization exists exactly once.

Every module references ClubID.

The Club Registry eliminates duplicate clubs, automatically links rosters, and becomes the backbone for Tournament Workspace, Scouting Matchup, Calendar and Team List.

3.2 Philosophy

A club is not simply a name.

It is a permanent basketball organization.

Names may change.

Sponsors may change.

Competitions may change.

The organization remains the same.

Everything should therefore reference ClubID instead of text.

Architecture

3.3 Club Lifecycle

Every club follows one lifecycle.

Figure 3.1 Workflow



The ClubID never changes.

3.4 Permanent Identity

Every club receives

ClubID

Examples

BC Vienna

Vienna

Basketball Club Vienna → ClubID 000347

Everything references this ID.

3.5 Club Object

Every club contains

Identity

ClubID

Official Name

Aliases

Historical Names

Abbreviation

Logo

Country

City

Website

Arena

Federation

Current Basketball

Current Competition

Current Season

Coach

Current Roster

Captain

League Position (future)

Historical Basketball

Previous Competitions

Previous Rosters

Historical Coaches

Historical Logos

Historical Names

Everything remains searchable.

3.6 Aliases [Key principle]

Every club supports unlimited aliases.

Example

Official

BC Vienna

Aliases

Vienna

Basketball Club Vienna

BCV

Searching any alias returns the same ClubID.

Importers also use aliases.

3.7 Historical Names

Sponsors change.

Club identities should not.

Example

Union Olimpija → Olimpija Ljubljana → Cedevita Olimpija

One ClubID.

Three historical names.

Nothing duplicated.

3.8 Matching Engine

Whenever importing clubs

DragonHub follows

Step 1

Exact Match → Done

Step 2

Alias Match → Done

Step 3

Fuzzy Match → Admin Confirmation → Remember forever

Step 4

Create Club

Only if no match exists.

3.9 Logo Management

Every club owns

One Official Logo.

Logo updates belong to Club Registry.

Every module automatically uses the same logo.

Dashboard

Calendar

Events

Scouting Matchup

Tournament Workspace

Reports

Player Profile

No module stores logos independently.

3.10 Roster Ownership [Key principle]

Current roster belongs to the Club Registry.

Not Tournament Workspace.

Not Scouting Matchup.

Not Calendar.

Club Registry owns

2026/27 → Roster

2025/26 → Roster

Historical rosters remain available.

3.11 Tournament Integration

Tournament → Import Teams → Club Registry → Current Roster → Players → Tournament Workspace

No manual player importing.

3.12 Scouting Matchup Integration

Scouting Matchup → Game → Home ClubID → Roster → Away ClubID → Roster

Everything comes directly from Club Registry.

No duplicated roster storage.

3.13 Calendar Integration

Calendar → Game → ClubID → Logo → Team Color → Roster

Everything shared.

3.14 Reports

Reports never store club names.

Reports reference

ClubID

Changing a club's official name automatically updates every report.

3.15 Club Timeline

Every club owns a Timeline.

Examples

Coach Changed

Roster Updated

Competition Changed

Report Added

Tournament Played

Everything chronological.

3.16 Team Notes

Every club supports Team Notes.

Example

```
Excellent spacing.  
Switch aggressively.  
Very physical guards.  
Weak transition defense.
```

Team Notes remain independent from Player Notes.

3.17 Club Profile

Every club eventually supports

Overview

Roster

Reports

Games

Timeline

History

Prospects

Everything opens inside a drawer.

Never another page.

3.18 Club Relationships

Club Registry automatically connects

Players

Competitions

Games

Reports

Events

Timeline

Fixtures

Statistics

Everything references ClubID.

3.19 Performance

Load order

Club → Logo → Roster → Games → Reports → Statistics

Never wait for statistics before displaying the roster.

Rules

3.20 Must Never Change

One ClubID.

Unlimited aliases.

Historical names preserved.

One official logo.

Roster belongs only to Club Registry.

Tournament Workspace consumes rosters.

Scouting Matchup consumes rosters.

Reports reference ClubID.

Everything references ClubID.

Approved Club Registry Principles

- One ClubID forever.
- Unlimited aliases.
- Historical names preserved.
- One official logo.
- One current roster.
- Historical rosters preserved.
- Every module consumes Club Registry.
- Tournament imports begin with clubs.
- Reports reference ClubID.
- Club Registry becomes the backbone of organizational data.

3.21 Success Criteria

Opening any club should immediately answer

- Who currently plays here?
- Which competition?
- What reports do we have?
- What games have we scouted?
- What do we know about this organization?
- How has it changed over time?

Without opening another module.

Architecture Notes

The Club Registry is the **organizational counterpart** to the Player Registry.

Where the Player Registry preserves the history of individual players, the Club Registry preserves the history of basketball organizations.

Together they create the two pillars of DragonHub's data model:

- **Player Registry** → Who the player is.
- **Club Registry** → Where the player belongs.

Once both registries are complete, almost every workflow we've designed - Tournament Workspace, Events, Scouting Matchup, Team List, Calendar, Reports and Player Profiles - becomes dramatically simpler because they all consume the same trusted club object instead of maintaining separate lists, logos and rosters.

I actually think this is where the Architecture Specification becomes more valuable than the Product Specification.

The Product Specification tells developers **what the platform should do**.

The Architecture Specification now starts telling them **how the platform should be built**.

I would expect Claude and Codex to read these architecture chapters before touching imports, registries, rosters or synchronization logic. They define the foundation that everything else should build upon.

Related chapters

[Chapter 2 - Player Registry](#)

[Chapter 4 - Competition Registry](#)

[Chapter 7 - Fixture Engine](#)

[Chapter 8 - Synchronization & Background Jobs Engine](#)

[Chapter 18 - Integration Layer & External Data Providers](#)

End of Chapter 3

Chapter 4

Competition Registry

4.1 Purpose

The Competition Registry is DragonHub's permanent registry of every basketball competition.

Every competition exists exactly once.

Every module references CompetitionID.

The Competition Registry standardizes:

- leagues,
- tournaments,
- national championships,
- youth competitions,
- showcases,
- international events.

4.2 Philosophy

Competitions should never be identified by text.

Competition names change.

Sponsors change.

Branding changes.

The basketball competition remains the same.

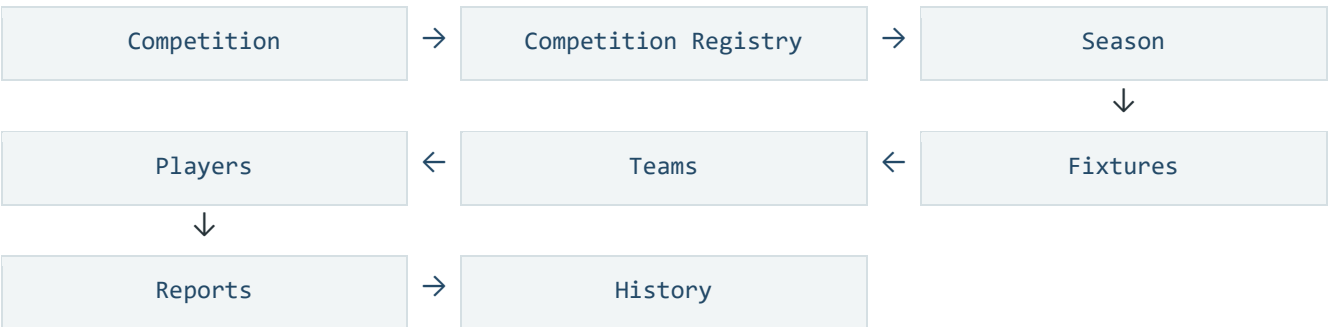
DragonHub therefore identifies competitions using

CompetitionID.

Architecture

4.3 Competition Lifecycle

Figure 4.1 Workflow



CompetitionID never changes.

4.4 Permanent Identity

Every competition receives

CompetitionID.

Examples

EuroCup

BKT EuroCup

```
EuroCup Basketball → CompetitionID 000071
```

Everything references this ID.

4.5 Competition Object

Every competition contains

Identity

CompetitionID

Official Name

Aliases

Historical Names

Logo

Country

Organizer

Website

Age Category

Competition Type

Gender

Professional / Youth

Current Season

Season

Current Teams

Fixtures

Standings

Statistics

Current Champion

Current Stage

Historical

Previous Seasons

Historical Winners

Historical Teams

Historical Logos

Historical Formats

4.6 Aliases

Unlimited aliases.

Example

Official

Basketball Champions League

Aliases

BCL

Champions League Basketball

Searching any alias always returns the same CompetitionID.

4.7 Historical Names

Sponsor changes should never create duplicates.

Example

ULEB Cup → EuroCup → BKT EuroCup

One CompetitionID.

Three historical names.

4.8 Competition Types

Every competition belongs to one type.

Examples

Professional League

Youth League

National Team

Cup

International Tournament

Showcase

Camp

Tryout

Friendly Tournament

Qualification Tournament

This allows filtering across DragonHub.

4.9 Season Model

Competition → contains

Seasons.

Example

EuroLeague → 2026/27 → 2025/26 → 2024/25

Statistics always belong to a season.

Not the competition itself.

4.10 Import Workflow

Competition import always happens first.

Approved pipeline

Competition → Competition Registry → Teams → Club Registry → Roster → Players → Statistics
→ Fixtures

Never skip Competition Registry.

4.11 Fixture Ownership

Fixtures belong to Competition Registry.

Not Club Registry.

Competition owns

Schedule

Round

Playoffs

Standings

Game Status

Everything competition-related remains here.

4.12 Statistics Ownership

Competition Registry defines

Statistics Source.

Examples

Official League → Official API → Import Schedule → Refresh Policy

The Statistics Engine reads this configuration.

4.13 Refresh Policy

Every competition defines

Refresh Frequency.

Examples

```
International Tournament → Daily  
EuroLeague → Every 3-4 days  
Domestic League → Weekly
```

No hardcoded schedules.

4.14 Event Integration

Events always reference CompetitionID.

Example

```
FIBA U16 → Competition Registry → Teams → Tournament Workspace → Reports
```

Everything connected.

4.15 Calendar Integration

Calendar also references CompetitionID.

Benefits

Logo

Competition Colour

Fixture Import

Standings

Everything automatic.

4.16 Reports

Reports never store competition names.

Reports reference CompetitionID.

Changing the competition name updates every report automatically.

4.17 Competition Timeline

Future support.

Examples

Season Started

Playoffs

Champion

Competition Format Changed

Everything chronological.

4.18 Competition Profile

Future drawer.

Overview

Standings

Fixtures

Teams

Reports

Timeline

Statistics

Everything remains connected.

4.19 Relationships

Competition Registry automatically connects

Clubs

Players

Games

Reports

Events

Timeline

Standings

Statistics

Fixtures

Everything references CompetitionID.

4.20 Performance

Priority

Competition → Fixtures → Standings → Teams → Statistics → Reports

Never wait for statistics before rendering fixtures.

Rules

4.21 Must Never Change

One CompetitionID.

Unlimited aliases.

Historical names preserved.

One statistics source.

One fixture source.

Everything references CompetitionID.

Competitions own fixtures.

Competitions own standings.

Approved Competition Registry Principles

- One CompetitionID forever.
- Unlimited aliases.
- Historical names preserved.
- Fixtures belong to Competition Registry.
- Standings belong to Competition Registry.
- Statistics source defined here.
- Refresh schedule defined here.
- Tournament Workspace begins from Competition Registry.
- Calendar consumes Competition Registry.
- Reports reference CompetitionID.

4.22 Success Criteria

Opening any competition should immediately answer

- Which teams participate?
- Which games are scheduled?
- Which reports do we have?
- Which players are being scouted?
- What statistics are available?
- Which events belong to this competition?

Without opening another module.

Architecture Notes

The **Competition Registry** completes the three foundational registries of DragonHub.

Together they form the permanent identity layer of the entire platform:

Player Registry → Club Registry → Competition Registry

Every basketball object in DragonHub should ultimately reference one or more of these three registries.

Once these registries are implemented correctly, modules no longer need to solve identity problems themselves. Tournament imports, Scouting Matchup, Calendar, Reports, Player Profiles and future modules all become simpler because they consume standardized objects instead of parsing names or maintaining duplicate records.

This architecture should remain unchanged throughout the lifetime of DragonHub because it provides the stable foundation upon which every future feature is built.

Related chapters

[Chapter 3 - Club Registry](#)

[Chapter 6 - Statistics Engine](#)

[Chapter 7 - Fixture Engine](#)

[Chapter 8 - Synchronization & Background Jobs Engine](#)

[Chapter 18 - Integration Layer & External Data Providers](#)

End of Chapter 4

Chapter 5

Relationship Engine

5.1 Purpose

The Relationship Engine is responsible for connecting every object inside DragonHub.

It ensures that observations, reports, games, players, clubs and competitions automatically know how they relate to each other.

The Relationship Engine removes manual linking.

Its purpose is to answer:

"What else is connected to this?"

5.2 Philosophy

DragonHub should behave like a basketball knowledge graph.

Information should never exist independently.

Instead of storing the same information multiple times, DragonHub stores relationships.

Example

```
Karl Chavis  
↓  
PlayerID  
↓  
Reports  
Games  
Events  
Tournament  
Watchlist  
Timeline  
Club  
Competition
```

The user should never manually recreate these connections.

Architecture

5.3 Relationship Model

Every object can relate to other objects.

Supported objects

```
Player
Club
Competition
Game
Event
Report
Video
User
```

Every relationship references IDs.

Never names.

5.4 Relationship Types

DragonHub supports several relationship types.

Examples

Player

belongs_to

Club

Player

plays_in

Competition

Player

participated_in

Game

Player

mentioned_in

Report

Player

observed_at

Event

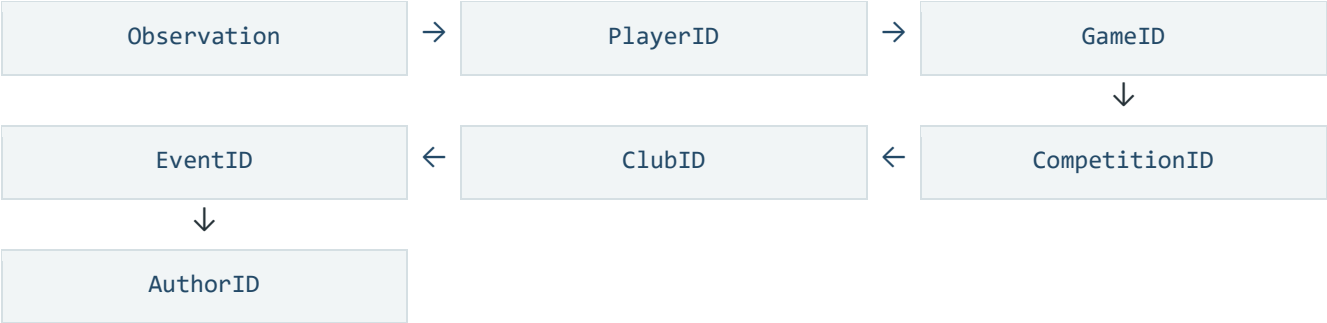
Everything is explicit.

5.5 Observation Relationships [Key principle]

Every observation automatically creates relationships.

Example

Figure 5.1 Workflow



Nothing else should be required.

5.6 Report Relationships

Reports may reference

One Player

Many Players

One Club

Many Clubs

One Competition

One Event

One Game

Everything references IDs.

Never duplicate report content.

5.7 Tournament Relationships

Tournament → Competition → Teams → Players → Observations → Reports → Timeline

Tournament Workspace should create these automatically.

5.8 Game Relationships

Game → Home Club

Away Club

Players

Reports

Scouting Matchup

Game Notebook

Timeline

Everything connected automatically.

5.9 Timeline Relationships

Timeline entries never contain duplicated information.

Instead

`Timeline → references`

ObservationID

ReportID

GameID

ClubID

CompetitionID

Timeline becomes an index.

Not storage.

5.10 Search Relationships

Global Search uses relationships.

Searching

Karl Chavis

returns

`Player → Reports → Games → Tournament → Club → Competition → Videos → Timeline`

No manual indexing.

5.11 Navigation

Every object should support contextual navigation.

Example

`Player → Club → Roster → Game → Tournament → Reports → Back`

The user should never reach a dead end.

5.12 Automatic Relationship Creation

Whenever the user creates

Observation

Report

Game

Tournament

Timeline

DragonHub automatically creates relationships.

Manual linking should be extremely rare.

5.13 Manual Relationships

Administrators may manually connect

Players

Clubs

Competitions

Reports

when automatic matching fails.

Manual relationships become permanent.

Future imports reuse them.

5.14 Relationship Priority

When multiple relationships exist

Priority

```
Player → Game → Competition → Club → Event → Report
```

This order determines contextual suggestions.

5.15 Relationship Validation

Every background import validates

PlayerID

ClubID

CompetitionID

```
Missing relationships → Warning
```

```
Duplicate relationships → Merge suggestion
```

Broken relationships should never remain hidden.

5.16 Relationship API

Every module should request information through relationships.

Example

Instead of

```
Get Reports
```

Request

```
Get Reports  
where PlayerID = X
```

The Relationship Engine resolves everything.

5.17 Performance

Relationships should be pre-indexed.

Opening a player should not trigger dozens of separate queries.

The engine should already know:

Reports

Games

Events

Videos

Timeline

Everything.

5.18 Future AI Integration

Artificial Intelligence should never search raw tables.

Instead

AI asks the Relationship Engine.

Examples

```
"Summarize Karl Chavis." → Relationship Engine → Reports → Timeline → Statistics → Scout Summary
```

Everything already connected.

Rules

5.19 Must Never Change

Relationships always reference IDs.

No duplicated information.

Automatic relationship creation is the default.

Navigation always follows relationships.

Timeline stores references rather than copies.

Search uses relationships.

Approved Relationship Engine Principles

- DragonHub behaves as a connected knowledge graph.
- Relationships replace duplicated data.
- Every relationship references IDs.
- Observations automatically create relationships.
- Reports automatically inherit relationships.
- Navigation follows relationships.
- Timeline indexes relationships.
- Search is relationship-aware.
- AI will consume relationships instead of raw tables.

5.20 Success Criteria

Opening any object inside DragonHub should immediately answer

- What is this?
- What is connected to it?
- What happened recently?
- Where else does it appear?

without manually searching.

Architecture Notes

I actually think the **Relationship Engine** is what separates DragonHub from almost every scouting platform I've seen.

Most systems are built around **tables**:

- Players table
- Reports table
- Games table

DragonHub should be built around **connections**.

That means the platform doesn't just know **what** something is - it knows **how it relates to everything else**.

This becomes the foundation for:

- automatic player timelines,
- report linking,
- tournament workspaces,
- watchlists,
- scouting history,
- global search,
- and eventually AI-assisted scouting.

If the Player Registry is the heart of DragonHub, then the Relationship Engine is its nervous system. Every future feature should plug into it rather than building isolated logic of its own. I honestly believe this architectural decision will be one of the reasons DragonHub feels fundamentally different from traditional basketball databases.

Related chapters

[Chapter 2 - Player Registry](#)

[Chapter 3 - Club Registry](#)

[Chapter 4 - Competition Registry](#)

[Chapter 13 - Event Bus & Workflow Engine](#)

[Chapter 16 - Knowledge Management Engine](#)

[Chapter 22 - Database Architecture & Schema Design](#)

End of Chapter 5

Part II - Core Engines

Chapter 6 Statistics Engine

Chapter 7 Fixture Engine

Chapter 8 Synchronization & Background Jobs Engine

Chapter 9 Intelligence Engine

Chapter 10 Global Search Engine

Chapter 11 Notification & Workflow Engine

Chapter 12 Component Engine & UI Framework

Chapter 13 Event Bus & Workflow Engine

Chapter 14 Data Flow & Lifecycle

Chapter 15 Feature Development Framework

Chapter 16 Knowledge Management Engine

Chapter 17 Workflow Automation Engine

[Architecture Diagram Placeholder]

Chapter 6

Statistics Engine

6.1 Purpose

The Statistics Engine is responsible for collecting, validating, updating and serving all statistical information inside DragonHub.

Its purpose is **not** to become a live scoring system.

Its purpose is to provide reliable basketball statistics that support scouting decisions.

Statistics should enhance scouting.

They should never replace scouting.

6.2 Philosophy

DragonHub is a scouting platform.

Not a statistics platform.

Statistics are supporting information.

Scouting reports remain the primary source of player evaluation.

Every statistics page should therefore answer

How current is this data?

before showing

What are the numbers?

Architecture

6.3 Ownership

Statistics are **not owned** by the Player Registry.

The Player Registry stores references only.

The Statistics Engine owns

- Current Season Stats
- Career Stats
- Game Logs
- Advanced Metrics
- Refresh History
- Sources

Everything statistical belongs here.

6.4 Statistics Object

Every statistics dataset contains

StatisticsID

PlayerID

CompetitionID

Season

Games

Minutes

Traditional Stats

Advanced Stats

Source

Last Updated

Refresh Schedule

Validation Status

Everything references IDs.

6.5 Statistics Sources

DragonHub supports multiple data providers.

Examples

Official League

Official Federation

Official Tournament

Licensed Provider

Manual Import

Every statistic always stores its source.

Never hide the origin of data.

6.6 Refresh Policy [Key principle]

Refresh frequency depends on competition type.

International tournaments

Refresh

Daily

Examples

FIBA

EuroBasket

EYBL

ANGT

Youth Championships

Reason

Games happen every day.

Major professional competitions

Refresh

Every 3-4 days

Examples

EuroLeague

EuroCup

ABA

BCL

ACB

LNB

BBL

Reason

Statistics change once or twice per week.

Domestic competitions

Refresh

Weekly

Examples

Lower divisions

Regional leagues

Youth leagues

Reason

Frequent updates are unnecessary.

Manual Refresh

Administrator may manually refresh

One player

One club

One competition

without waiting for scheduled synchronization.

6.7 Data Freshness

Every statistics page must display

Source → Last Updated → Refresh Schedule

Example

```
Source  
Official BCL  
Updated  
19 Sep 2026  
02:15  
Refresh  
Every 3 days
```

Users should never wonder how current the statistics are.

6.8 Statistics Priority

Statistics should always display

```
Current Season → Career → Advanced
```

Current season always appears first.

Scouts care about today's player.

Not five years ago.

6.9 Missing Statistics

Missing statistics are not treated as errors.

Instead display

```
No current statistics available.  
Last synchronization  
17 Sep 2026.
```

Never leave blank areas.

6.10 Validation

Before accepting imported statistics

Verify

PlayerID

CompetitionID

Season

ClubID

Games

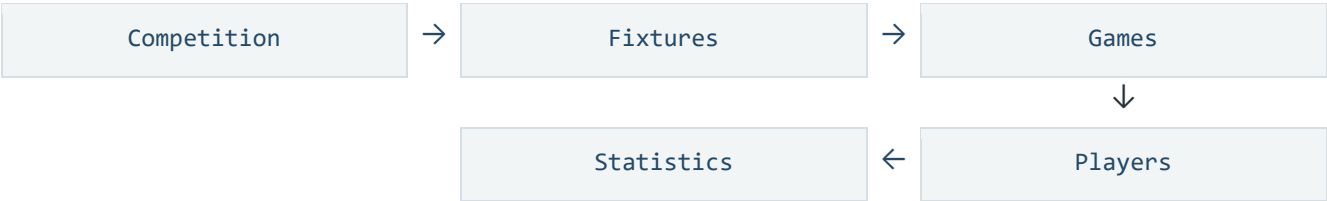
No duplicated statistics.

No orphan records.

6.11 Import Pipeline

Statistics import follows

Figure 6.1 Workflow



Never import statistics before rosters.

6.12 Statistics Cache

Frequently requested statistics should be cached.

Examples

Player Profile

Player Database

Watchlist

Review Queue

Shortlist

This avoids repeated calculations.

6.13 Advanced Metrics

The Statistics Engine supports

Efficiency

Usage

True Shooting

Assist Ratio

Rebounding %

Defensive Rating

Offensive Rating

Future advanced metrics belong here.

Not inside Player Registry.

6.14 Game Logs

Every imported game creates

Game Statistics → Player Statistics → Timeline → Reports

Everything becomes connected.

6.15 Performance

Priority

Player Identity → Scout Summary → Current Season Statistics → Career → Advanced Metrics → Game Logs

Never block the profile while advanced statistics load.

6.16 Relationship Integration

Statistics automatically connect to

Player Registry

Competition Registry

Club Registry

Timeline

Reports

Nothing is duplicated.

6.17 Future Analytics

Future modules may calculate

Development Curves

Trend Analysis

Rolling Averages

Competition Comparison

Projection Models

These calculations should always use the Statistics Engine.

Never duplicate statistics elsewhere.

Rules

6.18 Must Never Change

Statistics support scouting.

Not the opposite.

Refresh schedules remain competition-based.

Every statistics page displays

Source

Last Updated

Refresh Schedule

Player Registry never owns statistics.

Statistics Engine owns statistics.

Approved Statistics Engine Principles

- Statistics belong to the Statistics Engine.
- Competition-based refresh schedules.
- Daily tournaments.
- Every 3-4 days for major leagues.
- Weekly for smaller leagues.
- Source is always displayed.
- Last Updated is always displayed.
- Refresh Schedule is always displayed.
- Statistics remain secondary to scouting.
- Progressive loading is mandatory.

6.19 Success Criteria

A scout opening any player should immediately know

- What statistics are available.
- Where they came from.
- How current they are.
- Whether they are reliable.

Without needing to verify another website.

Architecture Notes

One of DragonHub's biggest strengths should be **transparency**.

Many basketball platforms display statistics without telling users:

- when they were updated,
- where they came from,
- or whether they are complete.

DragonHub should never hide this information.

Instead, every statistics page should communicate confidence:

"These numbers come from the Official ABA League, were refreshed two days ago, and follow a 3-day synchronization schedule."

That simple transparency builds trust and prevents scouts from making decisions based on stale or incomplete data. It also aligns perfectly with DragonHub's philosophy of supporting basketball decisions with reliable information rather than chasing unnecessary real-time updates. I think this is one of the architectural choices that will quietly make the platform feel much more professional.

Related chapters

[Chapter 2 - Player Registry](#)

[Chapter 4 - Competition Registry](#)

[Chapter 8 - Synchronization & Background Jobs Engine](#)

[Chapter 18 - Integration Layer & External Data Providers](#)

[Chapter 23 - API Architecture & Service Layer](#)

End of Chapter 6

Chapter 7

Fixture Engine

7.1 Purpose

The Fixture Engine is responsible for importing, synchronizing and distributing all scheduled basketball games inside DragonHub.

Its purpose is not simply to display fixtures.

Its purpose is to provide the correct games to the correct workflows.

The Fixture Engine answers:

"When is the next opportunity to watch this team or player?"

7.2 Philosophy

DragonHub is not a live scores application.

Fixtures exist to support planning.

The Fixture Engine therefore prioritizes

Planning → Scouting → Reports

instead of

Live Updates.

Architecture

7.3 Ownership

Fixtures belong to

Competition Registry.

Not

Player Registry.

Not

Club Registry.

The Fixture Engine simply distributes them.

7.4 Fixture Object

Every fixture contains

FixtureID

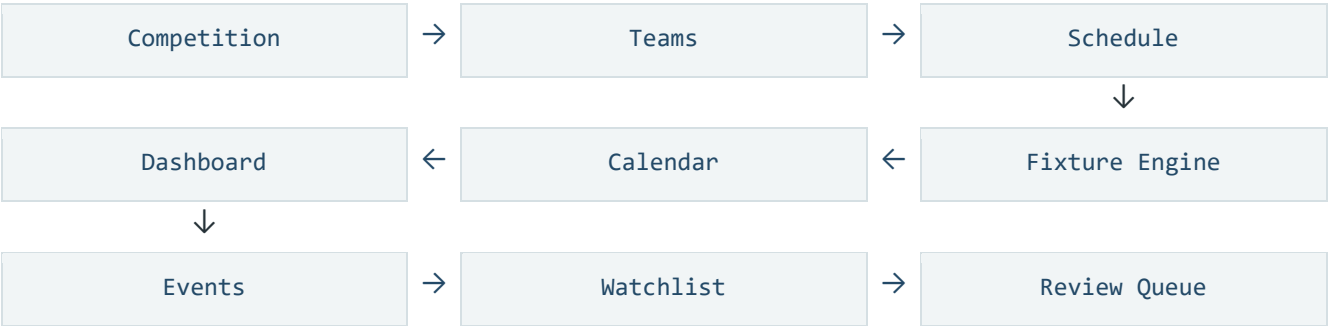
CompetitionID

Season
Round
Date
Time
Venue
Home ClubID
Away ClubID
Status
Officials (optional)
Attendance (future)
Game Report
Statistics Reference
Everything references IDs.

7.5 Import Pipeline

Fixtures always follow

Figure 7.1 Workflow



Never import fixtures before clubs exist.

7.6 Competition Ownership

Competitions own schedules.

Example

EuroLeague → Round 1 → Fixture → Fixture Engine

Never attach fixtures directly to clubs.

7.7 Club Distribution

Once fixtures exist

Fixture Engine automatically distributes

Club Schedule → Calendar → Dashboard → Roster → Game Overview

Everything updates automatically.

7.8 Watchlist Integration [Key principle]

Only tracked players receive fixtures.

Workflow

```
Player → Watchlist → Current Club → Fixture Engine → Next Game
```

Example

```
Tomorrow  
vs Partizan  
ABA League
```

The entire Player Database should **not** receive fixtures.

Only active scouting workflows.

7.9 Review Queue Integration

Review Queue also consumes

Next Game.

Example

```
Needs another viewing. → Tomorrow → Scout opportunity.
```

The queue becomes self-organizing.

7.10 Shortlist Integration

Shortlisted players also display

Next Game.

Useful before recruitment meetings.

7.11 Dashboard Integration

Dashboard consumes

Club Fixtures.

Approved widgets

Next Game

Upcoming Fixtures

Our Teams

Home Games

Recent Results

Everything comes from the Fixture Engine.

7.12 Calendar Integration

Calendar displays

Club fixtures only.

Events remain independent.

The Fixture Engine powers
Calendar.
Not Events.

7.13 Game Overview

Every fixture automatically creates
Game Overview.

[Overview](#) → [Game Notebook](#) → [Scouting Matchup](#) → [Reports](#) → [Timeline](#)

Everything starts from one FixtureID.

7.14 Results

When a fixture finishes

[Fixture](#) → [Result](#) → [Statistics](#) → [Timeline](#) → [Dashboard](#) → [Reports](#)

Everything updates automatically.

7.15 Fixture Status

Approved statuses

Scheduled

Live

Finished

Postponed

Cancelled

Archived

No custom statuses.

7.16 Refresh Policy

Fixtures should refresh

Daily

for active competitions.

[Major competitions](#) → Every 12-24 hours during the season.

[Lower competitions](#) → Every few days.

Never continuously.

7.17 Performance

Load order

[Fixture](#) → [Teams](#) → [Time](#) → [Logos](#) → [Officials](#) → [Additional metadata](#)

The schedule should always appear immediately.

7.18 Future

Future integrations

Travel

Arena Maps

Weather

TV Broadcast

Ticketing

Availability

These consume Fixture Engine.

Never duplicate fixtures.

Rules

7.19 Must Never Change

Fixtures belong to competitions.

Players consume fixtures.

Not the opposite.

Calendar remains club-focused.

Events remain scouting-focused.

Watchlist and Review Queue only display fixtures for active players.

Dashboard always consumes Fixture Engine.

Approved Fixture Engine Principles

- Fixtures belong to competitions.
- One FixtureID per game.
- Calendar consumes Fixture Engine.
- Dashboard consumes Fixture Engine.
- Watchlist receives fixtures only for active players.
- Review Queue receives fixtures only for active players.
- Progressive loading.
- No duplicate schedules.
- Results automatically update connected modules.

7.20 Success Criteria

Whenever a fixture changes,

DragonHub automatically updates

Dashboard

Calendar

Game Overview

Watchlist

Review Queue

Shortlist

Reports

Timeline

without manual intervention.

Architecture Notes

The Fixture Engine is **the scheduling backbone of DragonHub**.

It should never try to become a live-score provider.

Instead, it should provide the **right fixture to the right workflow**:

- **Calendar** → our club's operations.
- **Dashboard** → today's priorities.
- **Watchlist** → next opportunity to scout.
- **Review Queue** → next task.
- **Tournament Workspace** → tournament schedule.
- **Game Overview** → one game's operational hub.

By centralizing fixture ownership in one engine, every module benefits from consistent scheduling without maintaining its own copy of games. This also makes future additions - travel planning, arena information, TV coverage, or AI scouting recommendations - much easier because they all build on the same Fixture Engine rather than introducing parallel scheduling logic.

Related chapters

[Chapter 4 - Competition Registry](#)

[Chapter 8 - Synchronization & Background Jobs Engine](#)

[Chapter 13 - Event Bus & Workflow Engine](#)

[Chapter 17 - Workflow Automation Engine](#)

[Chapter 18 - Integration Layer & External Data Providers](#)

End of Chapter 7

Chapter 8

Synchronization & Background Jobs Engine

8.1 Purpose

The Synchronization Engine is responsible for keeping DragonHub up-to-date automatically.

Its responsibility is to synchronize:

- rosters,
- fixtures,
- statistics,
- profile photos,
- logos,
- automatic signals,
- timelines,
- relationships,

without requiring manual intervention.

The user should rarely need to think about synchronization.

8.2 Philosophy

Synchronization happens quietly.

The scout should never stop working because DragonHub is updating data.

All synchronization runs in the background.

Basketball work always takes priority over synchronization.

Architecture

8.3 Background Jobs

DragonHub consists of multiple independent background jobs.

Each job owns one responsibility.

Examples

Roster Sync

Fixture Sync

Statistics Sync

Logo Sync

Profile Photo Sync

Competition Sync

Signal Engine

Relationship Validation

Timeline Updates

Each job can run independently.

8.4 Job Scheduler

Every synchronization job contains

JobID

Job Name

Priority

Schedule

Status

Last Run

Next Run

Duration

Result

Retry Count

Error Log

Every job becomes monitorable.

8.5 Synchronization Priorities

Priority 1

Club Rosters → Everything else depends on them.

Priority 2

Competitions

Priority 3

Fixtures

Priority 4

Statistics

Priority 5

Profile Photos

Priority 6

Automatic Signals

Priority 7

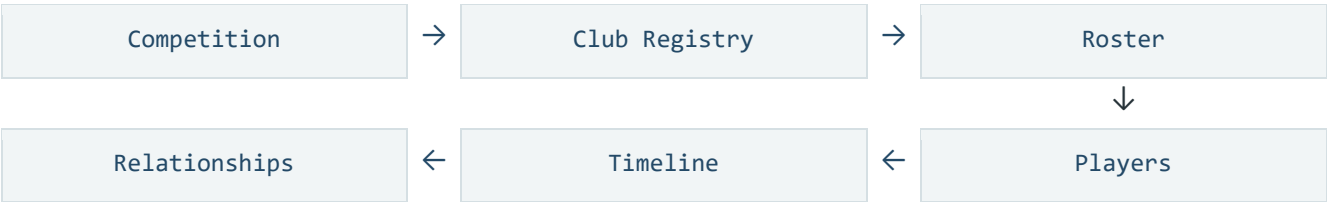
Relationship Validation

8.6 Roster Synchronization [Key principle]

Current season rosters are one of the most important jobs.

Approved behaviour

Figure 8.1 Workflow



Every roster update should automatically update the rest of DragonHub.

8.7 Statistics Synchronization

Uses

Statistics Engine.

Schedule

International tournaments → Daily
Major competitions → Every 3-4 days
Domestic competitions → Weekly

The schedule remains configurable.

8.8 Fixture Synchronization

Refreshes

Upcoming games

Results

Schedule changes

Cancelled games

Fixture Engine automatically distributes changes throughout DragonHub.

8.9 Profile Photo Synchronization

Profile photos should update automatically.

Workflow

Player → Photo available → Replace initials → Refresh everywhere

No manual refresh required.

8.10 Logo Synchronization

Every club logo belongs to Club Registry.

Once updated

Dashboard

Calendar

Reports

Tournament Workspace

Scouting Matchup

Player Profile

Everything immediately uses the new logo.

8.11 Automatic Signal Synchronization

Signals recalculate automatically after

Roster update

Competition change

Statistics update

Club transfer

No manual recalculation.

8.12 Timeline Synchronization

Every synchronization may create Timeline events.

Examples

Roster Updated

Competition Changed

Player Promoted

Statistics Updated

Club Changed

Timeline remains chronological.

8.13 Relationship Validation

After every synchronization

Relationship Engine validates

PlayerID

ClubID

CompetitionID

Report references

Missing references generate warnings.

8.14 Failure Recovery

Synchronization should never silently fail.

Approved workflow

Failure → Retry → Retry → Administrator Notification → Error Log

The user should never unknowingly work with incomplete data.

8.15 Manual Synchronization

Administrator supports

Refresh Player

Refresh Club

Refresh Competition

Refresh Statistics

Refresh Fixtures

Refresh Logos

Refresh Relationships

Everything independently.

8.16 Logging

Every synchronization creates

Start Time

Finish Time

Objects Updated

Objects Skipped

Warnings

Errors

Logs remain searchable.

8.17 Performance

Synchronization should never interrupt users.

No loading overlays.

No blocked pages.

Everything updates progressively.

8.18 Notifications

Only meaningful synchronization events generate notifications.

Examples

Roster import failed

Competition missing

Statistics outdated

Logo missing

Not

Successful synchronization

Users should not be spammed.

8.19 Future

Future synchronization jobs

Transfer Monitoring

Contract Monitoring

Player Availability

National Team Call-ups

Medical Status

Everything becomes another background job.

Rules

8.20 Must Never Change

Synchronization always runs in the background.

Users continue working.

Jobs remain independent.

One failed job should never stop another.

Everything automatically updates the Relationship Engine.

Approved Synchronization Principles

- Background jobs are independent.
- Users never wait for synchronization.
- Roster synchronization is highest priority.
- Statistics follow the approved refresh policy.
- Profile photos automatically replace initials.
- Logos are synchronized through Club Registry.
- Relationship validation follows every import.
- Failures are transparent.
- Manual synchronization remains available for administrators.

8.21 Success Criteria

DragonHub should automatically stay current.

The user should never need to ask

"Have the rosters been updated?"

or

"Are these statistics current?"

The system should already know.

Architecture Notes

This chapter defines the **heartbeat of DragonHub**.

If the registries are the **brain** and the Relationship Engine is the **nervous system**, then the Synchronization Engine is the **heartbeat**.

It keeps the platform alive without demanding attention from users.

As DragonHub grows, more background jobs will inevitably be added. Rather than creating custom synchronization logic inside each module, every automated process should become an independent job managed by this engine. That approach keeps the platform modular, easier to monitor, and far more resilient when one external data source changes or temporarily fails.

Related chapters

[Chapter 6 - Statistics Engine](#)

[Chapter 7 - Fixture Engine](#)

[Chapter 13 - Event Bus & Workflow Engine](#)

[Chapter 18 - Integration Layer & External Data Providers](#)

[Chapter 20 - Scalability, Infrastructure & Deployment](#)

End of Chapter 8

Chapter 9

Intelligence Engine

9.1 Purpose

The Intelligence Engine is responsible for automatically generating basketball intelligence from the data already stored inside DragonHub.

Unlike the Statistics Engine, which stores numbers, the Intelligence Engine creates meaning.

It answers one question:

What should the basketball department pay attention to?

The engine should reduce manual administration while increasing basketball awareness.

9.2 Philosophy

The Intelligence Engine never replaces scouts.

It highlights important information.

Scouts remain responsible for evaluations and decisions.

The engine exists to surface patterns that might otherwise be missed.

Architecture

9.3 Responsibilities

The Intelligence Engine owns:

- Automatic Signals
- Estimated Level suggestions (future)
- Estimated Ceiling suggestions (future)
- Next Scouting Opportunity
- Data freshness warnings
- Missing report detection
- Duplicate detection
- Recommendation suggestions
- Relationship validation warnings

It does **not** own statistics, reports or scouting opinions.

9.4 Automatic Signals [Key principle]

Signals are generated automatically.

Examples

Coming out of College

Rookie in Europe

First Overseas Season

Breakout Season

National Team Debut

Watchlist

High Upside

Top Prospect

Signals should never require manual entry.

9.5 Signal Rules

Example

Coming out of College

Condition

Previous Season → NCAA

Current Season → Professional Club

Signal → Coming out of College

Rookie in Europe

Condition

Previous Club → NBA

G League

NCAA

Current Club → Europe

Signal → Rookie in Europe

First Overseas Season

Condition

Country changed → First professional season outside home country.

Breakout Season

Condition

Minutes increased significantly.

Production increased significantly.

Signal generated.

9.6 Data Quality Signals

Examples

Profile photo missing.

Roster incomplete.

Statistics outdated.

Club not matched.

Competition not matched.

Missing report.

These are not shown to normal users.

Only administrators.

9.7 Watchlist Intelligence

When a player enters Watchlist

Automatically calculate

Last Seen

Next Game

Reports Available

Timeline Activity

No manual work.

9.8 Review Queue Intelligence

The engine should suggest

Needs another viewing.

Needs report.

Ready for discussion.

Ready for shortlist.

Everything remains suggestions.

Never automatic decisions.

9.9 Next Scouting Opportunity

Instead of simply displaying

Next Game

the engine determines

the next realistic scouting opportunity.

Examples

League game.

Tournament.

National Team.

Video available.

Tryout.

Future feature.

9.10 Timeline Intelligence

Timeline should detect

Long time since observation.

Rapid development.

Club transfer.

Competition change.

Promotion.

Everything remains informative.

9.11 Report Intelligence

Examples

Player has no report.

Tournament finished but no reports created.

Game notebook exists without report.

Duplicate reports.

Everything becomes suggestions.

9.12 Duplicate Detection

Examples

BC Vienna

Vienna → Suggest merge.

Player aliases → Suggest merge.

Competition aliases → Suggest merge.

Administrator confirms.

Never automatic.

9.13 Recommendation Engine

Future.

Examples

You have watched this player three times. → Generate Player Report.

Player not observed for six months. → Scout again.

Watchlist inactive. → Archive.

Everything remains optional.

9.14 Estimated Level Suggestions (Future)

DragonHub may eventually propose

Estimated Level

based on

Statistics

League

Competition

Reports

Age

Development

The scout always confirms manually.

9.15 Estimated Ceiling Suggestions (Future)

Youth Database → Projection → Suggested ceiling.

Manual confirmation required.

9.16 AI Integration

Artificial Intelligence consumes

Relationship Engine

Timeline

Reports

Statistics

Signals

Never raw database tables.

This ensures AI understands context.

9.17 Performance

The Intelligence Engine runs entirely in the background.

No blocking calculations.

Suggestions should already exist before the page opens.

Rules

9.18 Must Never Change

The Intelligence Engine never makes basketball decisions.

It highlights information.

Scouts evaluate.

Coaches decide.

Management approves.

The engine supports.

It never replaces people.

Approved Intelligence Engine Principles

- Intelligence supports scouts.
- Signals are automatic.
- Suggestions remain optional.
- No automatic basketball decisions.
- Data quality warnings remain administrator-only.
- AI consumes relationships rather than raw data.
- Estimated Level remains human-controlled.
- The platform becomes proactive, not autonomous.

9.19 Success Criteria

Opening DragonHub should automatically reveal

What changed?

What deserves attention?

What requires action?

Without manually searching.

The platform becomes proactive.

Architecture Notes

I actually think this chapter defines the difference between **a database** and **an operating system**.

A database waits for questions.

An operating system tells you what matters.

DragonHub should gradually evolve toward the second model.

For example, instead of making a scout remember that a Watchlist player is playing tomorrow, DragonHub should already know that and surface it. Instead of relying on memory to notice that a tournament has finished without any reports being created, the platform should gently point it out.

The Intelligence Engine therefore represents the beginning of DragonHub's transition from a passive information repository into an active basketball decision-support system. It should always remain **assistant-first**: offering insights, detecting patterns and reducing administrative work while leaving every basketball decision in human hands.

Related chapters

[Chapter 5 - Relationship Engine](#)

[Chapter 6 - Statistics Engine](#)

[Chapter 10 - Global Search Engine](#)

[Chapter 11 - Notification & Workflow Engine](#)

[Chapter 16 - Knowledge Management Engine](#)

[Chapter 17 - Workflow Automation Engine](#)

End of Chapter 9

Chapter 10

Global Search Engine

10.1 Purpose

The Global Search Engine is DragonHub's universal entry point.

Instead of navigating through multiple modules, the user should be able to find any basketball object from one search bar.

The purpose is simple:

Everything should be searchable.

The user should think about basketball.

Not about where information is stored.

10.2 Philosophy

Search should replace navigation whenever possible.

Instead of asking

"Which page is this on?"

the user simply searches.

DragonHub already knows where the information belongs.

Architecture

10.3 Search Scope

Global Search indexes

Players

Clubs

Competitions

Games

Events

Reports

Tournament Workspaces

Game Notebooks

Scouting Matchups

Teams

Users

Collections

Saved Views

Everything.

10.4 Search Bar

One search bar.

One behaviour.

Everywhere.

Example

Dashboard → Search

Player Database → Same Search

Reports → Same Search

Tournament Workspace → Same Search

The user never learns different search systems.

10.5 Search Behaviour

Typing

Walker

returns

Players → Leland Walker

Jordan Walker

Reports → Walker Report

Games → Walker vs Krka

Events → Walker Tryout

Grouped by category.

Not mixed together.

10.6 Search Categories

Results grouped into

Players

Clubs

Competitions

Games

Events

Reports

Users

Collections

Each category has its own icon.

10.7 Aliases [Key principle]

Search always understands aliases.

Example

Searching

Vienna

returns

BC Vienna

Searching

BCL

returns

Basketball Champions League

Searching

Miško

returns

Miško Ražnatović

Searching

Misko

returns

Miško Ražnatović

Everything uses Registry aliases.

10.8 Fuzzy Search

Minor spelling mistakes should still work.

Example

Houindo → Houido → Houndo

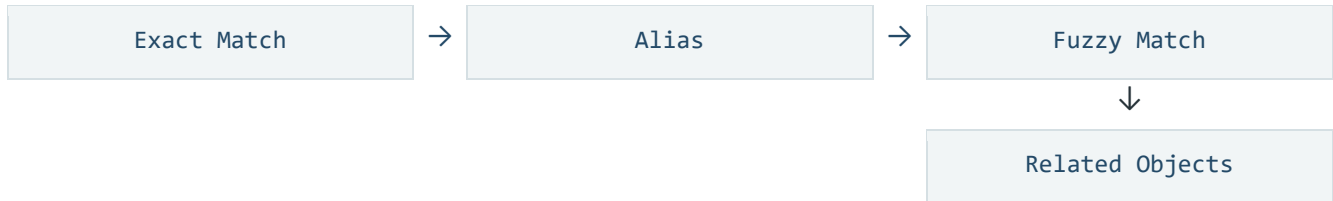
DragonHub should still suggest

Cameron Houindo.

10.9 Search Ranking

Priority

Figure 10.1 Workflow



Exact results always appear first.

10.10 Context Search

Search should understand where it is being used.

Example

Inside Tournament Workspace → Typing

Walker

searches

Tournament Roster

first.

Only afterwards

Youth Database → Pro Database → Everything else.

Exactly as we designed.

10.11 Player Search

Player results display

Photo

Name

Club

Nationality

Position

Estimated Level

Automatic Signals

One glance should identify the player.

10.12 Club Search

Club results display

Logo

Country

Competition

Current Roster Count

Everything required for quick identification.

10.13 Report Search

Reports display

Title

Report Type

Player

Date

Author

Collections

Click → Open Drawer.

Never another page.

10.14 Search Actions

Search results support

Open

Follow

Watchlist

Compare

Generate Report

Open Profile

Everything directly from search.

10.15 Keyboard Workflow

Global Search shortcut

Ctrl + K

(or configurable)

Search becomes the fastest navigation method inside DragonHub.

10.16 Performance

Search should feel instant.

Search Index updates automatically after

Roster Import

Player Import

Report Creation

Competition Import

Timeline Update

No manual indexing.

10.17 Future AI Search

Eventually users should be able to search naturally.

Examples

"Show EuroCup point guards under 25."

"Players we've watched three times."

"Tournaments in Spain next month."

The Relationship Engine powers these searches.

Rules

10.18 Must Never Change

There is only one search engine.

Every module consumes it.

Aliases remain universal.

Search replaces navigation whenever possible.

Context-aware search remains enabled.

Approved Search Engine Principles

- One search engine.
- One behaviour everywhere.
- Registry aliases supported.
- Context-aware search.
- Infinite indexing.
- Keyboard-first navigation.
- Search replaces navigation.
- Everything searchable.
- Search groups results by object type.

10.19 Success Criteria

A user should never need to ask

"Where is this stored?"

They simply search.

DragonHub already knows.

Architecture Notes

I actually think this chapter defines **how people will experience DragonHub five years from now.**

The more modules the platform gains - Contracts, Medical, Academy, Finance, Video Library - the less important navigation becomes.

Instead of remembering where something lives, users simply ask DragonHub.

That means the **Global Search Engine** gradually becomes the primary navigation method, while menus become secondary.

This also explains why we invested so much effort in the **Player Registry**, **Club Registry**, **Competition Registry**, and **Relationship Engine**. Those systems are what make a truly global search possible.

I honestly think that, long term, **Ctrl + K** (or the equivalent on iPad) will become one of the most-used features in DragonHub because it removes the need to think about where information is stored and lets basketball staff focus entirely on the task they're trying to accomplish.

Related chapters

[Chapter 2 - Player Registry](#)

[Chapter 3 - Club Registry](#)

[Chapter 4 - Competition Registry](#)

[Chapter 5 - Relationship Engine](#)

[Chapter 22 - Database Architecture & Schema Design](#)

[Chapter 23 - API Architecture & Service Layer](#)

End of Chapter 10

Chapter 11

Notification & Workflow Engine

11.1 Purpose

The Notification & Workflow Engine is responsible for notifying users about meaningful basketball events and guiding them toward their next action.

Notifications should never become noise.

They should answer one question:

"What deserves my attention right now?"

The engine supports basketball workflows rather than system events.

11.2 Philosophy

DragonHub should never behave like social media.

Notifications are not engagement tools.

Notifications exist only when they improve basketball decisions.

If a notification does not help the user perform their job, it should not be generated.

Architecture

11.3 Notification Categories

DragonHub supports four notification types.

Operational

Games

Calendar

Events

Practices

Travel

Scouting

Watchlist

Review Queue

Tournament

Reports

Scouting Matchup

Data

Roster Updates

Statistics

Missing Photos

Competition Imports

Club Matching

Administration

User Requests

Import Failures

Background Jobs

Permission Changes

11.4 Priority Levels

Every notification has one priority.

Critical → Immediate attention.

Examples

Import failed.

Club Registry mismatch.

Broken relationships.

Important → Today.

Examples

Watchlist player plays tonight.

Tournament starts tomorrow.

Report assigned.

Normal → This week.

Examples

Statistics refreshed.

Roster updated.

Silent → Timeline only.

No popup.

11.5 Notification Rules

Only generate notifications when
the user can take action.

Bad notification

"Roster synchronized."

No action.

Good notification

```
"Roster import failed." → Fix.
```

11.6 Watchlist Notifications [Key principle]

Examples

```
Next Game Tomorrow → Watchlist player → Scout opportunity.
```

```
Player moved clubs. → Review evaluation.
```

```
New report shared. → Read.
```

Everything directly supports scouting.

11.7 Review Queue Notifications

Examples

Needs second viewing.

Tournament finished.

Report missing.

Ready for discussion.

No generic reminders.

Only meaningful basketball workflow.

11.8 Tournament Notifications

Examples

Tournament begins tomorrow.

Rosters imported.

Games available.

Reports missing.

Tournament completed.

Every notification naturally moves the user toward Tournament Workspace.

11.9 Statistics Notifications

Normally hidden.

Only notify when

Statistics outdated.

Import failed.

Manual refresh completed.

Never notify successful background updates.

11.10 Club Registry Notifications

Examples

Duplicate club detected.

Alias suggested.

Missing logo.

Roster mismatch.

Administrator confirms.

Everything improves data quality.

11.11 Permission Notifications

Examples

User requested access.

Permission approved.

Permission revoked.

Role changed.

Administrator only.

11.12 Notification Delivery

Notifications appear in

Notification Center.

Optional Email.

Future Mobile Push.

Never duplicate notifications across channels unnecessarily.

11.13 Workflow Suggestions

DragonHub may recommend

Generate Player Report.

Archive Watchlist.

Move to Shortlist.

Scout Next Game.

Suggestions are optional.

Users remain in control.

11.14 Notification Center

Structure

Today

Yesterday

Earlier

Unread

Read

Search

Notifications become searchable.

11.15 Performance

Notifications are generated asynchronously.

Never delay page rendering.

Notification creation should not affect user interaction.

11.16 Future Automation

Future workflows

Automatic briefing.

Morning scouting summary.

Weekly watchlist review.

Tournament completion checklist.

Everything remains optional.

Rules

11.17 Must Never Change

Notifications support basketball.

Not engagement.

No meaningless alerts.

No notification spam.

Every notification should encourage a useful basketball action.

Approved Notification Principles

- Basketball-first notifications.
- Actionable only.
- No notification spam.
- Priority-based delivery.
- Watchlist and Review Queue remain the primary consumers.
- Background jobs remain mostly silent.
- Notification Center becomes searchable.
- Suggestions remain optional.

11.18 Success Criteria

A scout opening DragonHub should immediately understand

- What changed?
- What requires attention?
- What should I do next?

without checking multiple modules.

Architecture Notes

One thing we've learned while designing DragonHub is that **information alone doesn't improve decisions.**

Timing does.

A perfectly written report that the scout forgets to revisit before the player's next game has very little value.

The Notification & Workflow Engine exists to bridge that gap.

Its role is not to interrupt the user.

Its role is to quietly ensure that important basketball events - watchlist opportunities, tournament starts, missing reports, roster changes or review tasks - appear at the right moment.

The engine should therefore be viewed as the **assistant to the basketball department**, constantly organizing work in the background while leaving every basketball decision in human hands.

Related chapters

[Chapter 7 - Fixture Engine](#)

[Chapter 9 - Intelligence Engine](#)

[Chapter 13 - Event Bus & Workflow Engine](#)

[Chapter 17 - Workflow Automation Engine](#)

[Chapter 19 - Security, Permissions & Audit Architecture](#)

End of Chapter 11

Chapter 12

Component Engine & UI Framework

12.1 Purpose

The Component Engine defines every reusable UI component inside DragonHub.

No module should build its own buttons, tables, filters or cards.

Everything should be assembled from reusable components.

The objective is consistency.

12.2 Philosophy

DragonHub is one application.

Not twenty different websites.

The user should immediately recognize

Buttons

Cards

Tables

Drawers

Filters

Search

Popups

Everywhere.

The UI library becomes part of the architecture.

Architecture

12.3 Component Hierarchy

Components are divided into three levels.

Level 1

Foundation Components

Button

Icon

Chip

Badge

Avatar

Divider

Tooltip

Typography

Spacing

Everything else builds on these.

Level 2

Functional Components

Search Bar

Filter Row

Saved Views

Drawer

Modal

Dropdown

Table

Infinite List

Tabs

Notification

These are reused throughout DragonHub.

Level 3

Basketball Components [Key principle]

These are unique to DragonHub.

Player Card

Club Card

Competition Card

Game Card

Event Card

Player Header

Scout Summary

Quick Stats

Roster List

Timeline Card

Observation Card

These become DragonHub's identity.

12.4 Buttons

Approved button types

Primary
Green
Secondary
Grey
Danger
Red
Ghost
Transparent
No other button styles.

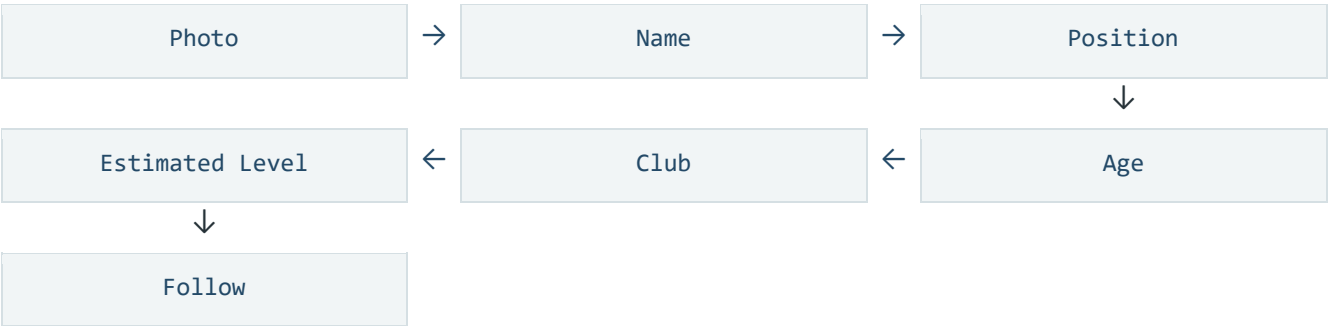
12.5 Chips

Purpose
Metadata only.
Examples
Pro
U18
EuroCup
Watchlist
Scouting
Online
Never use chips as primary actions.

12.6 Player Card

Every player card follows the same structure.

Figure 12.1 Workflow



Every module uses this component.
Player Database
Watchlist
Review Queue
Tournament Workspace

Team List

Free Agents

No custom versions.

12.7 Club Card

Every club card displays

Logo

Name

Competition

Country

Roster Size

Current Season

Everything references Club Registry.

12.8 Competition Card

Displays

Competition

Season

Country

Teams

Games

Status

Everything references Competition Registry.

12.9 Search Component

One search component.

Supports

Autocomplete

Aliases

Keyboard

Context Search

Every module uses the same implementation.

12.10 Filter Component

Every page uses the same filter order.

Search → Filters → Saved Views → Content

Never invent new layouts.

12.11 Drawer Component [Key principle]

Drawers become standard.

Examples

Player Profile

Club Profile

Reports

Statistics

Compare

Timeline

The underlying page always remains visible.

12.12 Infinite List

Large databases use

Infinite List.

Supported

Player Database

Youth Database

Watchlist

Review Queue

Team List

Free Agents

Never use page numbers.

12.13 Tables

Tables remain

Sortable

Searchable

Resizable

Sticky headers

Responsive

No custom table implementations.

12.14 Cards

Cards become the default container.

Approved

Rounded corners

Soft border

Minimal shadow

Consistent spacing

Cards should feel identical across DragonHub.

12.15 Empty States

Every empty page explains

Why it is empty.

What to do next.

Never display

"No data."

Example

```
No reports yet.  
Create your first Player Report.
```

Always guide the user.

12.16 Loading States

Skeleton loading becomes standard.

Never large blocking spinners.

Users should immediately understand what is loading.

12.17 Mobile Components

```
Desktop → iPad → Phone
```

The same components adapt.

Never create separate mobile interfaces.

12.18 Component Ownership

Every component exists once.

Updating

Player Card

automatically updates

Player Database

Watchlist

Review Queue

Tournament Workspace

Free Agents

No duplicated UI code.

Rules

12.19 Must Never Change

One Button.

One Search.

One Filter.

One Drawer.

One Player Card.

One Club Card.

One Event Card.

One Component Library.

Everything reuses these components.

Approved Component Engine Principles

- One component library.
- Basketball components become first-class objects.
- Player Card reused everywhere.
- Search reused everywhere.
- Drawers replace page navigation.
- Infinite List replaces pagination.
- Cards become standard containers.
- Skeleton loading everywhere.
- Components adapt rather than duplicate.

12.20 Success Criteria

Adding a new DragonHub module should require almost no new UI.

Developers simply assemble existing components.

The platform automatically remains consistent.

Architecture Notes

I actually think **this chapter is what separates DragonHub from becoming a collection of features.**

Most applications slowly accumulate dozens of slightly different buttons, cards, tables and filters as they grow.

DragonHub should actively prevent that.

Every new module - whether it's Contracts, Medical, Academy, Finance or Video Analysis - should first ask:

"Can I build this entirely using the existing DragonHub components?"

If the answer is yes, the platform remains visually and technically consistent.

That means future development becomes dramatically faster because developers stop inventing UI and instead focus on basketball functionality.

In the long run, I believe the **Component Engine** will save more development time than almost any other architectural decision we've documented because it protects DragonHub from the gradual UI fragmentation that affects so many long-lived software products.

Related chapters

[Chapter 10 - Global Search Engine](#)

[Chapter 15 - Feature Development Framework](#)

[Chapter 21 - Engineering Playbook](#)

[Chapter 23 - API Architecture & Service Layer](#)

[Chapter 24 - Quality Assurance, Testing & Release Management](#)

End of Chapter 12

Chapter 13

Event Bus & Workflow Engine

13.1 Purpose

DragonHub consists of many independent modules.

Player Registry

Club Registry

Statistics

Tournament Workspace

Reports

Calendar

Watchlist

Review Queue

Every module should remain independent.

The Event Bus allows them to communicate without becoming tightly coupled.

13.2 Philosophy

Modules should never directly update other modules.

Instead

they publish events.

Other modules react.

Example

```
Tournament Workspace → Observation Saved → Event Bus → Player Timeline → Reports →  
Watchlist → Review Queue
```

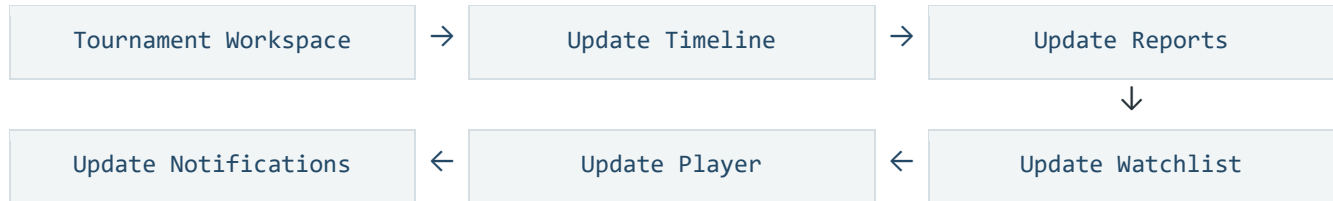
Everything updates automatically.

Architecture

13.3 Why Event Bus?

Without Event Bus

Figure 13.1 Workflow



One module now depends on five others.

This becomes impossible to maintain.

Instead

```

Tournament Workspace
↓
Publish
ObservationCreated
↓
Event Bus
↓
Everything else reacts
  
```

One responsibility.

13.4 Core Events

Examples

PlayerCreated

PlayerUpdated

ClubUpdated

RosterUpdated

FixtureUpdated

StatisticsUpdated

ObservationCreated

ReportCreated

TimelineCreated

WatchlistAdded

ReviewQueueAdded

ShortlistAdded

Everything becomes an event.

13.5 Example Workflow [Key principle]

Scout writes

Tournament Observation.

Tournament Workspace publishes

```

ObservationCreated
  
```

Automatically

```
Timeline → Update  
Player Profile → Update  
Watchlist → Update  
Reports → Update  
Search Index → Update
```

Notifications

No manual code.

Everything reacts automatically.

13.6 Report Workflow

User creates

Player Report.

Event

```
ReportCreated
```

Automatically

```
Player Timeline → Update  
Player Profile → Update  
Club Profile → Update  
Competition Profile → Update  
Search → Update
```

Everything connected.

13.7 Roster Workflow

```
Background Job → Roster Updated → Publish  
RosterUpdated
```

Automatically

```
Team List → Tournament Workspace → Scouting Matchup → Calendar → Dashboard → Player  
Registry
```

Everything refreshed.

13.8 Statistics Workflow

```
Statistics Engine → StatisticsUpdated → Player Profile → Database → Watchlist → Review  
Queue → Shortlist
```

Everything receives updated statistics.

13.9 Fixture Workflow

Fixture Engine → FixtureUpdated → Dashboard → Calendar → Watchlist → Review Queue → Game Overview

No duplicated scheduling logic.

13.10 Notification Workflow

Example

Player moved clubs.

Publish

ClubChanged

Notification Engine decides

Should anyone be notified?

The source module never knows.

13.11 Search Workflow

New Report. → Publish

SearchIndexUpdate

Search Engine updates automatically.

No manual indexing.

13.12 Timeline Workflow

Everything eventually creates Timeline events.

Examples

Observation

Report

Roster Update

Competition Change

Club Transfer

Watchlist

Review Queue

Timeline becomes automatic.

13.13 Event Naming

Every event follows

Object
Verb

Examples

PlayerCreated

ClubUpdated

ReportDeleted

ObservationCreated

Never use vague names.

13.14 Event Rules

Events contain

ObjectID

Timestamp

Author

Metadata

Everything references IDs.

Never names.

13.15 Error Handling

If one module fails → Other modules continue.

Example

Statistics update failed.

Timeline still updates.

Reports still exist.

The Event Bus isolates failures.

13.16 Future AI

Future AI

doesn't watch modules.

AI watches events.

Example

PlayerObserved → AI suggests

Generate Report.

Everything becomes event-driven.

13.17 Performance

Events should be asynchronous.

Never block the UI.

The user continues scouting.

Background updates happen automatically.

Rules

13.18 Must Never Change

Modules never call each other directly.

Everything communicates through events.

Every important action becomes an event.

Events reference IDs.

The Event Bus remains the communication backbone.

Approved Event Bus Principles

- Modules remain independent.
- Events connect the system.
- One action → one event.
- Event Bus distributes updates.
- Timeline reacts automatically.
- Search reacts automatically.
- Notifications react automatically.
- Reports react automatically.
- Modules never directly update one another.

13.19 Success Criteria

Adding a new module should require almost no modifications to existing modules.

The new module simply subscribes to existing events.

DragonHub grows without increasing coupling.

Architecture Notes

I honestly think this chapter is **the biggest difference between software that survives five years and software that survives twenty.**

Without an Event Bus, every new feature adds more direct connections between modules. Over time, changing one page breaks five others.

With an Event Bus, DragonHub becomes **event-driven.**

A module only needs to know **what happened**, not **who needs the information**.

That means future modules - Transfer Tracker, Medical, Academy, Finance, AI Assistant, Contract Center - can simply subscribe to existing events instead of rewriting core logic.

I actually think this architecture is what will allow DragonHub to continue growing for years without becoming fragile. It keeps modules independent while making the whole platform feel seamlessly connected.

Related chapters

[Chapter 5 - Relationship Engine](#)

[Chapter 8 - Synchronization & Background Jobs Engine](#)

[Chapter 11 - Notification & Workflow Engine](#)

[Chapter 17 - Workflow Automation Engine](#)

[Chapter 23 - API Architecture & Service Layer](#)

End of Chapter 13

Chapter 14

Data Flow & Lifecycle

14.1 Purpose

This chapter defines how information travels through DragonHub.

Every piece of data should have one clear lifecycle.

The objective is to prevent duplicated logic, duplicated storage and inconsistent updates.

Data should always move in one direction.

14.2 Philosophy

DragonHub is **event-driven**.

Data should never randomly appear inside modules.

Instead, every piece of information follows a predictable lifecycle.

Example

```
Competition → Club → Roster → Player → Observation → Report → Timeline → Recruitment
```

Every developer should immediately know where data comes from and where it goes next.

Architecture

14.3 Data Categories

DragonHub stores three different kinds of information.

Public Data

Imported.

Examples

Players

Clubs

Competitions

Fixtures

Statistics

Standings

Logos

Profile Photos

Internal Basketball Intelligence

Created by scouts.

Examples

Reports

Observations

Estimated Level

Scout Summary

Watchlist

Review Queue

Shortlist

Tags

System Data

Created automatically.

Examples

Timeline

Relationships

Signals

Notifications

Background Jobs

Everything generated by DragonHub itself.

14.4 Data Ownership

Every type of data has one owner.

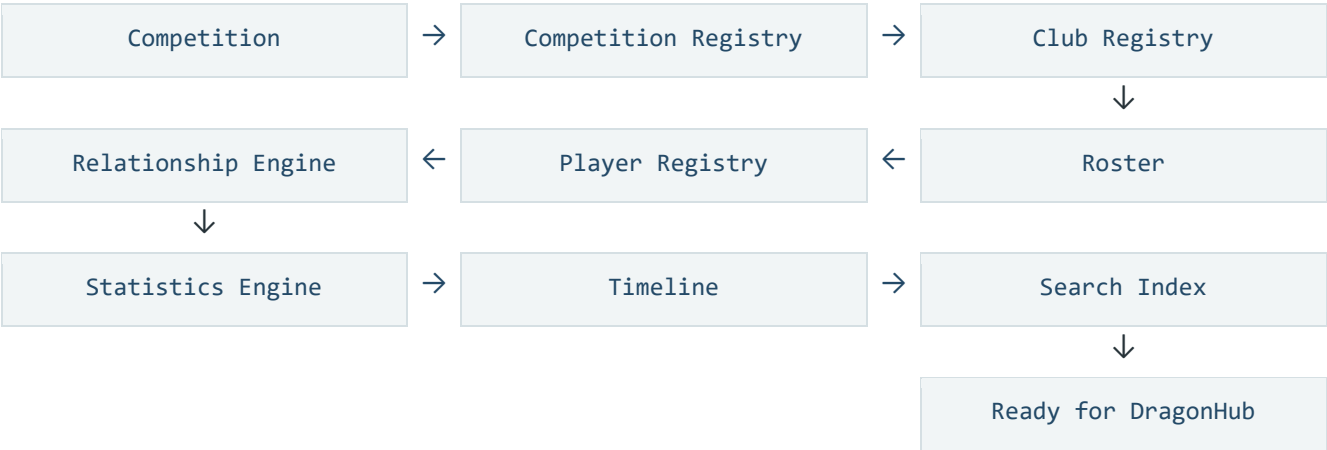
Data	Owner
Players	Player Registry
Clubs	Club Registry
Competitions	Competition Registry
Fixtures	Fixture Engine
Statistics	Statistics Engine
Reports	Reports Module
Observations	Game Notebook / Tournament Workspace
Timeline	Timeline Engine
Relationships	Relationship Engine
Notifications	Notification Engine

No shared ownership.

14.5 Data Lifecycle [Key principle]

Every imported player follows the same path.

Figure 14.1 Workflow



No shortcuts.

No alternative paths.

14.6 Observation Lifecycle

This is the most important workflow in DragonHub.

Scout → Observation → ObservationID → Relationship Engine → Timeline → Player Profile → Reports → Watchlist → Review Queue → Shortlist

The observation is written once.

Everything else is automatic.

14.7 Report Lifecycle

Reports never own observations.

Reports reference them.

Workflow

Observation → Player Report → ReportID → Player Profile → Timeline → Search → Collections

Reports summarize observations.

They never duplicate them.

14.8 Club Lifecycle

Competition → Club Registry → Roster → Players → Games → Reports → Timeline

Every club remains connected throughout its history.

14.9 Player Promotion

Youth → Professional → Retired

PlayerID never changes.

The player's category changes.

Not the player.

14.10 Deletion Rules

DragonHub should almost never delete data.

Instead

Archive.

Examples

Archived Reports

Archived Players

Archived Clubs

Archived Events

Historical information remains valuable.

14.11 Versioning

Important objects support history.

Examples

Scout Summary

Estimated Level

Reports

Club Roster

Every important change should be recoverable.

14.12 Audit Trail

Every modification stores

User

Timestamp

Previous Value

New Value

Reason (optional)

Especially for

Estimated Level

Scout Summary

Reports

Player merges

Club merges

14.13 Synchronization Flow

Every synchronization follows

```
Import → Validation → Registry Matching → Relationship Update → Timeline → Search Index → Notifications
```

Everything follows the same sequence.

14.14 Manual Changes

Manual edits always override imported data.

Example

```
Imported Height → Scout manually corrects height → Future imports should not overwrite it without confirmation.
```

DragonHub should respect manual basketball knowledge.

14.15 Conflict Resolution

If imported data conflicts with manual data

Display

Imported Value

Manual Value

Choose

Keep Manual

Accept Import

Merge

Administrator decides.

14.16 Data Expiration

Some information naturally expires.

Examples

Current Club

Current Competition

Availability

Next Game

Everything historical remains.

Only current values change.

14.17 Performance

Data should always move

Background → Validation → Registry → Relationships → UI

The interface never waits for the full pipeline.

Rules

14.18 Must Never Change

Data always follows one lifecycle.

Observations are never duplicated.

Reports reference observations.

Registries own identities.

Engines own processes.

Modules consume information.

Everything follows one direction.

Approved Data Flow Principles

- One owner for every piece of data.
- One lifecycle for every object.
- Observations are written once.
- Reports summarize observations.
- Registries own identities.
- Engines own processes.
- Modules consume data.
- Archive instead of delete.
- Manual basketball knowledge always has priority over imported public data.

14.19 Success Criteria

A developer should be able to answer

Where does this data originate?

Who owns it?

Who updates it?

Who consumes it?

How does it expire?

within seconds.

If not,

the architecture is too complex.

Architecture Notes

I honestly think this chapter is what will keep DragonHub maintainable **10 years from now**.

The biggest problem in long-lived sports software isn't writing new features - it's losing track of **where data comes from** and **who is responsible for it**.

By explicitly defining:

- ownership,
- lifecycle,
- synchronization,
- conflict resolution,
- and expiration,

DragonHub avoids the gradual chaos that usually appears after years of development.

This chapter should become mandatory reading for every backend developer because almost every future feature - whether it's contracts, medical data, academy management or AI - will eventually plug into the same data flow rather than inventing a new one. That's what will keep DragonHub coherent as it grows.

Related chapters

Chapter 2 - Player Registry

Chapter 5 - Relationship Engine

Chapter 16 - Knowledge Management Engine

Chapter 19 - Security, Permissions & Audit Architecture

Chapter 22 - Database Architecture & Schema Design

End of Chapter 14

Chapter 15

Feature Development Framework

15.1 Purpose

Every new DragonHub feature should follow the same development framework.

The objective is consistency.

Developers should never start coding immediately.

Every feature follows the same lifecycle.

This prevents redesigns that break existing workflows.

15.2 Philosophy

DragonHub grows through evolution.

Not through reinvention.

Every new feature should integrate into existing architecture instead of creating parallel systems.

Whenever possible:

Reuse.

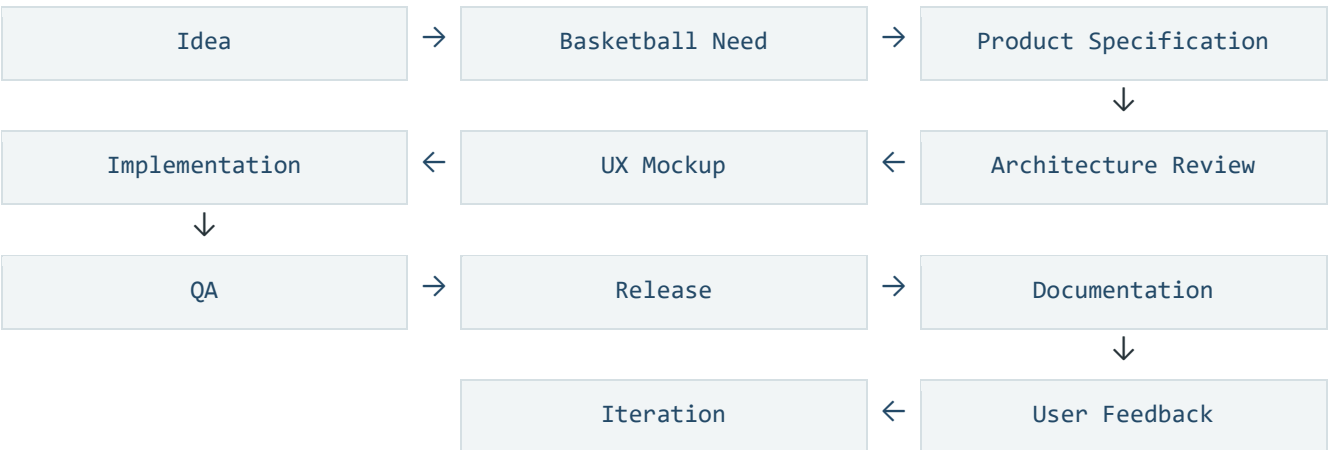
Never duplicate.

Architecture

15.3 Development Lifecycle

Every feature follows this sequence.

Figure 15.1 Workflow



No feature skips Product Specification.

15.4 Step 1 - Basketball Need

Every feature begins with a basketball problem.

Never a technical problem.

Good example

"Scouts spend too much time manually searching tournament players."

Bad example

"Let's build another page."

Technology always follows basketball.

15.5 Step 2 - Product Specification

Before coding begins, define:

Purpose

Workflow

Users

Expected behaviour

Success criteria

If this is missing,

the feature is not ready.

15.6 Step 3 - Architecture Review

Ask:

Which existing systems already solve part of this?

Examples

Player Registry

Club Registry

Competition Registry

Relationship Engine

Statistics Engine

Timeline Engine

Event Bus

If the answer is

`"already exists" → Reuse it.`

15.7 Step 4 - UX Review

Before implementation ask

Can an existing DragonHub component solve this?

Examples

Drawer

Player Card

Search

Filter

Table

Saved View

Timeline Card

Never create new UI unless necessary.

15.8 Step 5 - Implementation

Implementation should follow architecture.

Never invent:

New Player model

New Club model

New Competition model

Everything consumes existing engines.

15.9 Step 6 - QA [Key principle]

QA consists of two parts.

Functional QA

Does it work?

Basketball QA

Does this actually improve basketball workflow?

The second question is equally important.

15.10 Regression Checklist

Every feature must verify

Dashboard

Calendar

Events

Tournament Workspace

Game Notebook

Reports

Player Database

Player Profile

Watchlist

Review Queue

Shortlist

Nothing should break.

15.11 Documentation

No feature is complete until

DragonHub Guide

is updated.

Every feature ships with

Screenshot

Workflow

FAQ

Keyboard shortcuts

if applicable.

Documentation is part of development.

15.12 Release Readiness

Before release ask

Does it match Product Specification?

Does it reuse architecture?

Does it reuse components?

Does it improve basketball workflow?

Would Marko immediately understand why it exists?

If any answer is

"No" → Do not release.

15.13 Feature Categories

DragonHub supports three types of features.

Type A

Infrastructure

Examples

Club Registry

Relationship Engine

Statistics Engine

High impact.

No visible UI.

Type B

Workflow

Examples

Tournament Workspace

Review Queue

Watchlist

Highest product value.

Type C

Visual

Examples

Buttons

Spacing

Animations

Lowest priority.

Visual polish should never delay infrastructure.

15.14 Prioritization

Future work follows

Data → Workflow → Automation → Performance → Visual Polish

Good data always creates better software than better colors.

15.15 Technical Debt

Every release should reduce technical debt.

Never increase it.

Examples

Merge duplicated logic.

Remove duplicate queries.

Reuse components.

Reuse registries.

Refactor before extending.

15.16 Experimental Features

Future experimental ideas

AI

Medical

Finance

Travel

Contracts

should remain isolated until validated.

Core architecture should remain stable.

15.17 Code Reviews

Every large feature should be reviewed against
Product Specification
Architecture Specification
Design System
Developer Handbook
before merging.

Rules

15.18 Must Never Change

Product first.
Architecture second.
Implementation third.
Never reverse this order.
No feature bypasses Product Specification.
No feature bypasses Architecture.

Approved Development Framework

- Basketball problem first.
- Product Specification first.
- Architecture review second.
- UX review third.
- Implementation fourth.
- QA fifth.
- Documentation sixth.
- Release last.
- Data before UI.
- Workflow before visual polish.
- Reuse before redesign.

15.19 Success Criteria

A new developer joining DragonHub should know exactly how to implement a feature before writing code.
The process should be predictable.
Repeatable.
Scalable.

Architecture Notes

Looking back at everything we've worked on over the past months, almost every major regression happened because one of these steps was skipped.

For example:

- The **Scouting Matchup** redesign happened without checking the original product workflow.
- **Tournament Workspace** initially duplicated player selection instead of using the Club Registry.
- **BC Vienna vs Vienna** happened because imports bypassed a central identity layer.
- UI inconsistencies appeared when components were redesigned instead of reused.

This framework is designed to prevent exactly those situations. Every future feature - whether it's a new AI assistant, Transfer Center, Medical module or Academy tools - should pass through the same sequence before implementation. That keeps DragonHub growing in a controlled, consistent way instead of accumulating disconnected solutions. I actually think this chapter will become one of the most practical parts of the Architecture Specification because it defines not just **how DragonHub works**, but **how DragonHub should continue to evolve**.

Related chapters

Chapter 12 - Component Engine & UI Framework

Chapter 21 - Engineering Playbook

Chapter 24 - Quality Assurance, Testing & Release Management

Chapter 25 - Decision Framework & Product Governance

Chapter 26 - Engineering Constitution

End of Chapter 15

Chapter 16

Knowledge Management Engine

16.1 Purpose

The Knowledge Management Engine is responsible for preserving, organizing and surfacing the basketball knowledge created inside DragonHub.

Unlike public data, scouting knowledge belongs to the club forever.

The engine answers one question:

What has the club already learned about this player, team or competition?

16.2 Philosophy

DragonHub should never lose knowledge.

Every observation should remain valuable.

Even if

- a scout leaves,
- a coach changes,
- ten years pass,

the knowledge remains inside DragonHub.

Knowledge compounds over time.

Architecture

16.3 Knowledge Sources

The engine automatically collects information from

Game Notebook

Tournament Workspace

Scouting Matchup

Player Reports

Video Reports

Tryout Reports

Manual Notes

Timeline

Everything contributes.

16.4 Knowledge Object

Every knowledge item contains

KnowledgeID

Source

Author

Timestamp

PlayerID

ClubID

CompetitionID

GameID

EventID

Tags

Visibility

Everything references IDs.

16.5 Knowledge Types

Approved types

Observation

Summary

Report

Recommendation

Projection

Decision

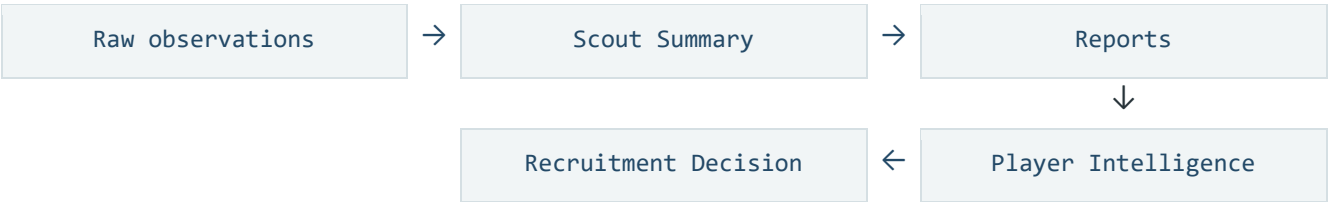
Meeting Note

Internal Comment

Everything remains searchable.

16.6 Knowledge Hierarchy [Key principle]

Figure 16.1 Workflow



DragonHub should never confuse
raw observations
with

final conclusions.

16.7 Scout Summary

Scout Summary becomes
the current opinion.

Reports remain historical.

Example

```
Player → 10 reports → Scout Summary → Current recommendation
```

The Summary evolves.

Reports never change.

16.8 Historical Knowledge

Nothing should overwrite history.

Example

```
2024 → Development Player
```

```
2025 → EuroCup Prospect
```

```
2026 → EuroLeague Rotation
```

All three remain visible.

DragonHub preserves the evolution of opinions.

16.9 Tags

Knowledge supports

Basketball Tags

Examples

Shooter

Playmaker

Rim Protector

High Motor

Transition

PnR

Decision Making

Tags remain basketball-focused.

16.10 Search

Knowledge becomes searchable.

Examples

```
"Rim Protector" → All observations
```

Reports

Players

Searching should span the entire knowledge base.

16.11 Knowledge Timeline

Every player automatically builds

Knowledge Timeline.

Example

```
2024
Tournament Observation
↓
2025
Player Report
↓
2026
Scouting Matchup
↓
2026
Tryout
↓
2027
Recruitment
```

Everything chronological.

16.12 Recommendations

Knowledge should always distinguish

```
Observation → Recommendation
```

Observation

Example

"Excellent passer."

Recommendation

"Suitable for EuroCup."

These are different.

16.13 Versioning

Scout Summary supports versions.

Previous summaries remain archived.

Current Summary remains active.

No knowledge should disappear.

16.14 Knowledge Ownership

Observations belong to

Authors.

Knowledge belongs to

DragonHub.

When a scout leaves the organization,
their reports remain part of the club's institutional memory.

16.15 Visibility

Knowledge supports permissions.

Private

Department

Club

Specific Users

Visibility is independent from ownership.

16.16 Future AI

AI should consume

Knowledge Engine

instead of raw reports.

Example

```
Summarize player. → Knowledge Engine → Scout Summary → Timeline → Reports → Recommendations
```

This produces much better results.

16.17 Performance

Knowledge should be indexed.

Searching

Players

Tags

Reports

Recommendations

should remain instant.

Rules

16.18 Must Never Change

Knowledge is never deleted.

Only archived.

Scout Summary represents the current opinion.

Reports preserve historical opinions.

Observations remain immutable.

Knowledge belongs to the organization.

Not individual scouts.

Approved Knowledge Management Principles

- Knowledge compounds over time.
- Nothing is deleted.
- Reports preserve history.
- Scout Summary reflects today's opinion.
- Observations remain immutable.
- Knowledge belongs to the club.
- Tags remain basketball-focused.
- AI consumes knowledge rather than raw reports.
- Every player builds a permanent knowledge timeline.

16.19 Success Criteria

A scout opening any player should immediately understand

- What have we learned?
- How has our opinion changed?
- Which reports support that opinion?
- What should we do next?

Without reading years of reports individually.

Architecture Notes

I honestly think this chapter defines DragonHub's **true competitive advantage**.

Most scouting software stores reports.

DragonHub should store **knowledge**.

There's an important difference:

- A report is a document.
- Knowledge is the accumulation of every observation, discussion, recommendation and decision over years.

That means DragonHub isn't just remembering *what* was written - it understands **how the club's opinion evolved**.

Imagine opening a player in 2032 and seeing:

- first observation at age 14,
- tournament reports,
- scouting matchup notes,
- promotion to the Pro Database,

- recruitment discussions,
- final signing decision.

That is far more valuable than a folder full of disconnected PDFs. I think this engine is what transforms DragonHub from a scouting database into the **institutional basketball memory** of the club, which is exactly the long-term vision we've been building throughout these specifications.

Related chapters

[Chapter 2 - Player Registry](#)

[Chapter 5 - Relationship Engine](#)

[Chapter 14 - Data Flow & Lifecycle](#)

[Chapter 19 - Security, Permissions & Audit Architecture](#)

[Chapter 22 - Database Architecture & Schema Design](#)

End of Chapter 16

Chapter 17

Workflow Automation Engine

17.1 Purpose

The Workflow Automation Engine connects basketball workflows together.

Instead of asking the user

"What do you want to do next?"

DragonHub should already know the next logical step.

The engine never makes basketball decisions.

It automates basketball administration.

17.2 Philosophy

Basketball workflows should feel continuous.

Users should never repeatedly perform the same administrative actions.

Whenever possible

DragonHub performs them automatically.

Basketball decisions always remain human.

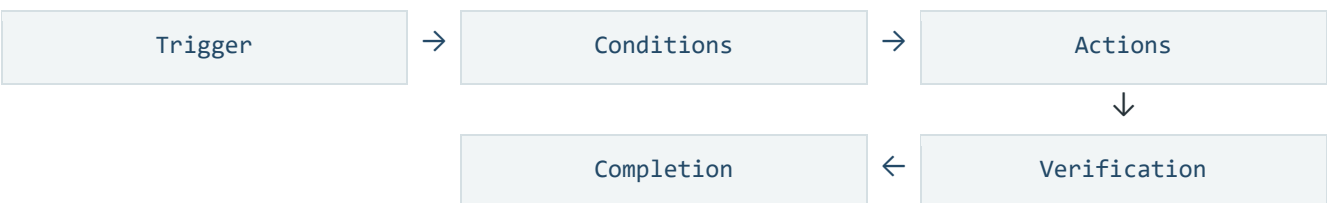
Administrative work becomes automated.

Architecture

17.3 Workflow Definition

Every workflow consists of

Figure 17.1 Workflow



Every workflow remains visible and traceable.

17.4 Example Workflow [Key principle]

Tournament imported. → Teams imported. → Club Registry. → Current Rosters. → Tournament Workspace ready. → No manual setup.

17.5 Observation Workflow

Scout writes observation. → Save.

Automatically

Relationship Engine → Timeline → Knowledge Engine → Player Profile → Search Index → Reports
→ Notifications

The scout writes once.

Everything else happens automatically.

17.6 Watchlist Workflow

Player added to Watchlist.

Automatically

Track Next Game. → Track Last Seen. → Enable Statistics Monitoring. → Enable Timeline
Monitoring. → Available for Review Queue.

No additional setup.

17.7 Review Queue Workflow

Conditions

Enough observations. → Player Report missing. → Generate Review Task.

Example

Needs Player Report.

Everything generated automatically.

17.8 Shortlist Workflow

Conditions

Player Report completed. → Enough observations. → Discussion completed. → Ready for
Shortlist.

DragonHub suggests.

Scouts decide.

17.9 Report Workflow

Tournament finished. → Generate suggested Player Reports. → Prefill observations. → Ready
for editing.

No copy-pasting.

17.10 Promotion Workflow

Youth Player → Professional Contract.

Automatically

Move category

```
Youth → Professional
```

PlayerID remains.

Timeline remains.

Reports remain.

Everything connected.

17.11 Club Workflow

```
Roster updated. → Club Registry. → Player Registry. → Tournament Workspace. → Scouting  
Matchup. → Calendar. → Dashboard.
```

Everything updates automatically.

17.12 Competition Workflow

```
Competition imported. → Competition Registry. → Fixtures. → Clubs. → Rosters. → Statistics.  
→ Ready.
```

17.13 Data Quality Workflow

```
Duplicate detected. → Administrator Review. → Merge. → Remember.
```

Future imports become automatic.

17.14 Report Suggestions

Examples

```
Tournament complete. → Generate Player Reports.  
Player observed three times. → Generate Scout Summary.  
Watchlist inactive. → Suggest Archive.
```

Suggestions remain optional.

17.15 Notification Workflow

```
Workflow Engine → Notification Engine
```

The Workflow Engine never displays notifications directly.

It requests them.

Notification Engine decides.

17.16 Workflow Ownership

Each workflow has one owner.

Examples

```
Roster Workflow → Club Registry.  
Statistics Workflow → Statistics Engine.  
Timeline Workflow → Relationship Engine.
```

Avoid duplicated workflow logic.

17.17 Manual Override

Users may override

Workflow Suggestions.

Overrides are respected.

Automation never forces actions.

17.18 Workflow Logging

Every workflow records

Started

Completed

Skipped

Failed

Retried

Everything becomes auditable.

17.19 Performance

Workflow execution always runs

Background → Event Bus → Relationship Engine → Notification → UI Update

Never interrupt scouting.

17.20 Future Automation

Possible future workflows

Transfer monitoring

Contract expiry

National Team selection

Academy promotion

Medical clearance

Everything follows the same framework.

Rules

17.21 Must Never Change

Automation removes administration.

Not basketball decisions.

Suggestions remain optional.

Scouts remain responsible.

Workflows always remain transparent.

Approved Workflow Automation Principles

- Automation supports workflows.
- Decisions remain human.
- Workflows are transparent.
- Every workflow has one owner.
- Automation is optional where judgment is required.
- Administrative work is minimized.
- Basketball remains central.
- Background execution only.
- Workflow logs remain available.

17.22 Success Criteria

A scout should gradually notice that

DragonHub is doing more and more administrative work automatically while requiring fewer manual actions every season.

The platform should feel increasingly intelligent without becoming unpredictable.

Architecture Notes

I actually think this chapter captures the biggest long-term ambition for DragonHub.

Most software waits for users to click buttons.

DragonHub should quietly prepare the next step before the user even asks.

Not by making decisions for the basketball department - but by removing the repetitive administrative work that surrounds those decisions.

For example:

- Tournament imported → rosters already linked.
- Observation saved → timeline already updated.
- Player promoted → databases already synchronized.

- Report completed → Review Queue already updated.

The scout should gradually feel like DragonHub is working alongside them rather than simply storing information.

That's the difference between a database and a true basketball operating system.

Related chapters

[Chapter 5 - Relationship Engine](#)

[Chapter 8 - Synchronization & Background Jobs Engine](#)

[Chapter 9 - Intelligence Engine](#)

[Chapter 11 - Notification & Workflow Engine](#)

[Chapter 13 - Event Bus & Workflow Engine](#)

End of Chapter 17

Part III - Infrastructure

Chapter 18 Integration Layer & External Data Providers

Chapter 19 Security, Permissions & Audit Architecture

Chapter 20 Scalability, Infrastructure & Deployment

[Architecture Diagram Placeholder]

Chapter 18

Integration Layer & External Data Providers

18.1 Purpose

The Integration Layer is responsible for every communication between DragonHub and external systems. Its purpose is to isolate DragonHub from third-party providers.

If an external provider changes its API or website, only the Integration Layer should require modification. Every other module remains unchanged.

18.2 Philosophy

DragonHub should never directly depend on external websites.

Instead

```
DragonHub → Integration Layer → External Provider
```

Everything passes through one abstraction layer.

Architecture

18.3 Responsibilities

The Integration Layer manages

- Statistics
- Fixtures
- Rosters
- Club Information
- Competition Information
- Profile Photos
- Logos
- Standings
- External Links

Nothing else should communicate directly with external providers.

18.4 Provider Architecture

Every provider is implemented as an independent connector.

Example

Figure 18.1 Workflow



Another example

```
Official EuroLeague → EuroLeague Connector → Integration Layer → DragonHub
```

Providers never communicate directly with DragonHub modules.

18.5 Connector Standard

Every connector implements the same interface.

Required functions

```

Connect()
Validate()
Import()
Normalize()
Match()
Update()
Log()
Disconnect()
  
```

Regardless of provider.

18.6 Supported Data Types

Each connector declares what it supports.

Examples

Roster

Fixtures

Statistics

Standings

Player Photos

Club Logos

Competition Data

No connector should import data outside its declared responsibility.

18.7 Provider Independence [Key principle]

DragonHub should never know whether data came from

Official League

Licensed Provider

Federation

Manual Import

Everything should appear identical once imported.

The Statistics Engine, Fixture Engine and Registries consume normalized data only.

18.8 Data Normalization

Every imported object is normalized before entering DragonHub.

Examples

Different providers may return

```
BC Vienna  
Vienna  
Basketball Club Vienna
```

The Integration Layer never passes raw names forward.

```
Instead → Club Registry → ClubID
```

Everything downstream receives standardized objects.

18.9 Import Sequence

Every provider follows the same order.

```
Competition → Teams → Club Registry → Roster → Players → Fixtures → Statistics →  
Relationships → Timeline
```

Never bypass the sequence.

18.10 Provider Metadata

Every imported object stores

Provider

Import Time

Import Version

Validation Status

Confidence

This improves debugging and transparency.

18.11 Manual Providers

DragonHub also supports manual imports.

Examples

CSV

Excel

JSON

Future APIs

Manual imports still pass through the same normalization pipeline.

No shortcuts.

18.12 Failover

If one provider becomes unavailable

DragonHub should

```
Retry → Fallback Provider (if configured) → Administrator Notification → Continue functioning
```

The rest of DragonHub must remain operational.

18.13 Provider Priority

When multiple providers exist

Priority order determines ownership.

Example

```
Official League → Federation → Licensed Provider → Manual Import
```

Higher-priority providers override lower ones unless a manual override exists.

18.14 Manual Overrides

Administrators may permanently override imported data.

Examples

Correct player height.

Correct club.

Correct competition.

Correct profile photo.

Future imports should respect these manual corrections unless explicitly approved.

18.15 Logging

Every connector logs

Import Started

Import Finished

Objects Imported

Objects Updated

Objects Skipped

Warnings

Errors

Everything remains searchable.

18.16 Monitoring

The Integration Layer continuously monitors

Connection Health

Import Duration

Import Failures

Missing Data

Synchronization Delay

Administrators should immediately know if a provider stops working.

18.17 Security

No provider should have direct database access.

Every import passes through

Validation → Normalization → Registry Matching → Relationship Engine → Database

Nothing bypasses this pipeline.

18.18 Future Providers

The architecture should support adding future connectors without modifying DragonHub.

Examples

Official Federation

League API

Licensed Statistics Provider

Video Platform

Transfer Platform

Everything becomes another connector.

Rules

18.19 Must Never Change

DragonHub never depends directly on providers.

Every provider implements the same interface.

Every import passes through normalization.

Registries always resolve identities.

Modules never consume raw provider data.

Approved Integration Principles

- One Integration Layer.
- Provider-independent architecture.
- One connector per provider.
- Normalization before storage.
- Registries resolve identities.

- Manual overrides are respected.
- Provider failures never stop DragonHub.
- Logging and monitoring are mandatory.
- Modules never consume raw external data.

18.20 Success Criteria

Adding a completely new league should require

Creating one connector.

Configuring refresh schedules.

Mapping fields.

No module should require modification.

DragonHub should scale by adding connectors rather than changing architecture.

Architecture Notes

I actually think this chapter protects DragonHub from one of the biggest risks long-term sports software faces: **dependency on individual data providers**.

Today you might use one source for rosters, another for statistics, and a third for fixtures. Five years from now those providers may have changed completely.

By forcing every provider through the same Integration Layer, DragonHub becomes **provider-agnostic**. The rest of the system doesn't care where the data came from - it only receives validated, normalized basketball objects.

That means replacing or adding a provider becomes an engineering task measured in days rather than a platform-wide rewrite. I think this architectural separation is essential if DragonHub is going to remain maintainable and adaptable over the next decade.

Related chapters

[Chapter 3 - Club Registry](#)

[Chapter 4 - Competition Registry](#)

[Chapter 6 - Statistics Engine](#)

[Chapter 7 - Fixture Engine](#)

[Chapter 8 - Synchronization & Background Jobs Engine](#)

[Chapter 23 - API Architecture & Service Layer](#)

End of Chapter 18

Chapter 19

Security, Permissions & Audit Architecture

19.1 Purpose

DragonHub stores the basketball department's most valuable asset:

Knowledge.

Unlike public basketball websites, DragonHub contains confidential scouting reports, internal evaluations, recruitment strategies and long-term planning.

The purpose of this architecture is to ensure that this knowledge remains:

- Secure
- Traceable
- Recoverable
- Permission-controlled

at all times.

19.2 Philosophy

Security should never reduce usability.

Users should only see the information they need.

The platform should feel simple while remaining secure.

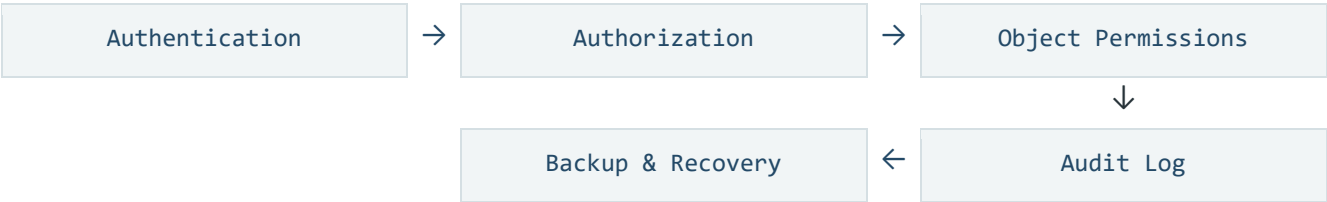
Every permission should exist because it protects basketball knowledge - not because it makes the software more complicated.

Architecture

19.3 Security Layers

DragonHub security consists of five layers.

Figure 19.1 Workflow



Each layer operates independently.

19.4 Authentication

Authentication answers

Who is this user?

Supported methods

Email

Google (future)

Microsoft (future)

Club SSO (future)

Two-Factor Authentication (future)

Authentication should never contain business logic.

19.5 Authorization

Authorization answers

What is this user allowed to do?

Permissions are role-based.

Roles never contain business data.

Roles only determine access.

19.6 Role Hierarchy

```
Administrator → Professional Scout → Youth Scout → Front Office → Coach → Youth Coach → Viewer
```

Every role inherits only the permissions explicitly assigned to it.

19.7 Object Permissions [Key principle]

Every important object supports its own visibility.

Examples

Report

Player

Collection

Event

Notebook

Media

Visibility options

Private

Specific Users

Department

Club

Everyone

Object permissions override general role permissions where appropriate.

19.8 Audit Trail

Every meaningful modification is recorded.

Examples

Report Edited

Estimated Level Changed

Scout Summary Updated

Club Merge

Player Merge

Roster Override

Statistics Override

Everything remains traceable.

19.9 Audit Record

Each audit entry contains

AuditID

UserID

Timestamp

ObjectID

ObjectType

Action

Old Value

New Value

Reason (optional)

Nothing important changes silently.

19.10 Manual Overrides

Manual basketball knowledge has priority.

Examples

Scout corrects

Height → Imported data should not overwrite it.

Examples

Estimated Level

Scout Summary

Club Match

Player Match

Manual changes remain protected.

19.11 Data Recovery

Nothing important should be permanently deleted.

Instead

Soft Delete → Archive → Recovery

Reports

Observations

Scout Summaries

Collections

Everything recoverable.

19.12 Version History

Support version history for

Scout Summary

Reports

Estimated Level

Manual Notes

Club Notes

Users should be able to review previous versions.

19.13 Merge History

Player Merge

Club Merge

Competition Merge

All merge operations should remain reversible whenever technically possible.

19.14 Confidential Data

Examples

Scout Recommendations

Recruitment Discussions

Internal Ratings

Contract Notes

Medical (future)

These should never be publicly exposed.

Visibility always requires explicit permission.

19.15 Background Jobs

Background jobs should never bypass permissions.

Examples

Statistics Update

Roster Import

Timeline Update

Relationship Validation

Jobs update data.

They never change visibility.

19.16 Logging

Every security-related event is logged.

Login

Logout

Failed Login

Permission Change

Role Change

Import Override

Manual Correction

Everything searchable.

19.17 Backup Strategy

Automatic backups

Daily

Weekly

Monthly

Support full recovery.

Support object-level recovery where possible.

19.18 Disaster Recovery

DragonHub should recover

Players

Clubs

Competitions

Reports

Observations

Relationships

Timeline

without losing basketball knowledge.

Backups are more important than statistics.

Statistics can be re-imported.

Scouting knowledge cannot.

Rules

19.19 Security Principles

Protect

Scout Reports

Scout Summaries

Recruitment Notes

Internal Ratings

These are the club's intellectual property.

Everything else can be recreated.

19.20 Must Never Change

Reports are never permanently deleted.

Manual scouting knowledge always has priority.

Every important action remains auditable.

Permissions always remain object-based.

Backups always include scouting knowledge.

Approved Security Principles

- Basketball knowledge is the most valuable asset.
- Authentication identifies users.
- Authorization grants permissions.
- Object permissions protect confidential information.
- Every important action is auditable.
- Manual knowledge always overrides imported data.
- Reports are archived rather than deleted.
- Scout Summaries support version history.
- Statistics are replaceable.
- Scouting knowledge is not.

19.21 Success Criteria

A basketball club should trust DragonHub with its entire scouting department.

Even years later the club should be able to answer
Who wrote this?
When?
Why was it changed?
What did it previously contain?
Nothing important should ever become untraceable.

Architecture Notes

This chapter reflects one of DragonHub's core assumptions:

The club's scouting knowledge is more valuable than its public data.

If a statistics provider disappears tomorrow, DragonHub can rebuild the statistics.

If five years of scouting reports disappear, that knowledge is gone forever.

Because of that, the architecture intentionally treats:

- reports,
- observations,
- timelines,
- recommendations,
- scout summaries,

as the most protected information in the entire platform.

Everything else - including statistics, fixtures and imported rosters - can always be reconstructed.

That distinction should guide every future security decision and every backup strategy. DragonHub is ultimately protecting the basketball department's accumulated knowledge, and that should remain the platform's highest security priority.

Related chapters

[Chapter 14 - Data Flow & Lifecycle](#)

[Chapter 16 - Knowledge Management Engine](#)

[Chapter 20 - Scalability, Infrastructure & Deployment](#)

[Chapter 22 - Database Architecture & Schema Design](#)

[Chapter 23 - API Architecture & Service Layer](#)

End of Chapter 19

Chapter 20

Scalability, Infrastructure & Deployment

20.1 Purpose

DragonHub is designed as a long-term basketball platform.

The architecture should comfortably support

- multiple clubs,
- multiple databases,
- decades of scouting history,
- millions of records,

without requiring architectural redesign.

Scalability is considered part of the original design rather than an afterthought.

20.2 Philosophy

DragonHub should grow by

adding data,

not by rewriting architecture.

Every new competition.

Every new country.

Every new season.

Every new module.

should integrate into the existing platform.

Not replace it.

Architecture

20.3 Scalability Targets

DragonHub should comfortably support

Players

500,000+

Clubs

25,000+

Competitions

5,000+

Reports

5,000,000+

Observations

50,000,000+

Games

2,000,000+

without architectural changes.

20.4 Modular Architecture

Every subsystem remains independent.

Examples

Player Registry

Club Registry

Competition Registry

Statistics Engine

Fixture Engine

Relationship Engine

Timeline Engine

Knowledge Engine

Notification Engine

Workflow Engine

Every module scales independently.

20.5 Database Philosophy

The database should prioritize

Relationships

over

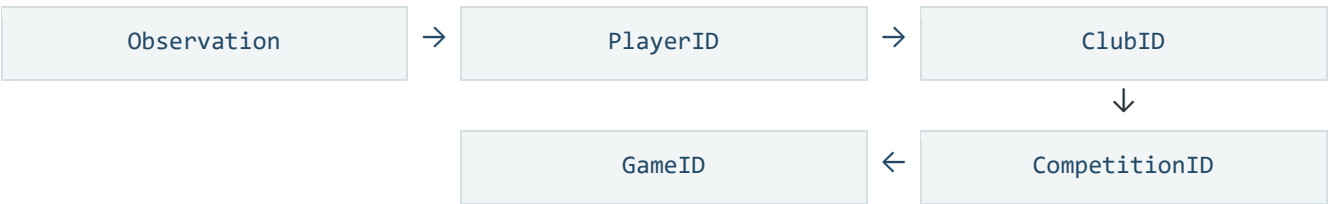
duplication.

Every object should be referenced.

Not copied.

Example

Figure 20.1 Workflow



Instead of embedding repeated information.

20.6 Storage Strategy

Separate storage by responsibility.

Examples

Relational Database → Players

Clubs

Competitions

Relationships

Timeline

Object Storage → Photos

Logos

Videos

Documents

PDFs

Search Index → Global Search

Autocomplete

Knowledge Search

Cache → Frequently accessed information

Never mix responsibilities.

20.7 Caching

Cache should exist on multiple levels.

Examples

Profile Photos

Club Logos

Competition Logos

Statistics

Search Results

Saved Views

Dashboard

Calendar

Frequently used data should almost never require a fresh database query.

20.8 Background Processing

Heavy work should never run in the user's request.

Examples

Roster Imports

Statistics Imports

Relationship Validation

Signal Generation

Search Index Updates

Timeline Rebuilding

Everything runs asynchronously.

20.9 API Layer

Every client communicates through one API.

Desktop → API → Business Logic → Database

iPad → Same API

Phone → Same API

Future Mobile App → Same API

Business rules never belong in the UI.

20.10 Versioning

Every API should support versioning.

Examples

v1

v2

v3

Breaking changes should never affect existing clients unexpectedly.

20.11 Multi-tenancy (Future)

Long-term

DragonHub should support multiple clubs.

Each club owns

Players

Reports

Watchlists

Scouting

Permissions

Data remains isolated.

Architecture should allow this from day one.

20.12 Monitoring

Every major subsystem should be monitored.

Examples

Database

API

Imports

Background Jobs

Search

Notifications

Storage

Performance

Failures

Everything observable.

20.13 Logging

Logs should be categorized.

Application

Database

Imports

Security

API

Background Jobs

Everything searchable.

20.14 Release Strategy

Three environments.

Development → Testing → Production

No direct production development.

Every feature passes through QA.

20.15 Deployment

Deployments should be

Incremental

Rollback capable

Zero downtime where practical

Every deployment should be reversible.

20.16 Disaster Recovery

Support

Point-in-time recovery

Object recovery

Database recovery

Media recovery

Registry recovery

Basketball knowledge remains the highest priority.

20.17 Performance Goals

Player Profile

< 1 second

Player Search

Instant

Dashboard

< 1 second

Tournament Workspace

< 2 seconds

Scouting Matchup

Immediate

These become engineering targets.

20.18 Technical Debt

Every release should reduce

or maintain

technical debt.

Never knowingly increase it.

Refactor continuously.

Never postpone architectural cleanup indefinitely.

Rules

20.19 Must Never Change

Architecture scales through

Registries

Relationships

Engines

Modules

Never through duplicated logic.

Performance remains a product feature.

Every subsystem remains independent.

Everything communicates through architecture rather than shortcuts.

Approved Infrastructure Principles

- Modular architecture.
- Independent engines.
- Relationship-first database.
- Heavy work runs in background jobs.
- One API for every client.
- Cache aggressively.
- Scale horizontally through architecture.
- Zero duplication.
- Performance remains measurable.
- Basketball knowledge remains the highest-value data.

20.20 Success Criteria

Five years from now

DragonHub should still feel like

one application

despite

millions of observations,

hundreds of thousands of players,

multiple clubs,

multiple databases,

and decades of basketball knowledge.

The architecture should allow growth without redesign.

Architecture Notes

I actually think this chapter captures one of the biggest differences between **building software** and **building a platform**.

Most applications are designed for today's users.

DragonHub should be designed for the basketball department you'll have **ten years from now**.

That means the architecture should assume:

- more scouts,

- more clubs,
- more countries,
- more competitions,
- more reports,
- more observations,

without changing its foundations.

If the registries, engines and relationship model remain stable, DragonHub should be able to grow almost indefinitely by adding data rather than rewriting code.

That, in my opinion, is the defining characteristic of a true basketball operating system rather than a collection of scouting tools.

Related chapters

[Chapter 8 - Synchronization & Background Jobs Engine](#)

[Chapter 18 - Integration Layer & External Data Providers](#)

[Chapter 19 - Security, Permissions & Audit Architecture](#)

[Chapter 23 - API Architecture & Service Layer](#)

[Chapter 24 - Quality Assurance, Testing & Release Management](#)

End of Chapter 20

Part IV - Platform Foundation

Chapter 21 Engineering Playbook

Chapter 22 Database Architecture & Schema Design

Chapter 23 API Architecture & Service Layer

Chapter 24 Quality Assurance, Testing & Release Management

[Architecture Diagram Placeholder]

Chapter 21

Engineering Playbook

21.1 Purpose

This chapter defines the engineering standards for every DragonHub developer.

It explains

- how features should be implemented,
- how code should be organized,
- how modules should communicate,
- how quality should be maintained.

This chapter exists to ensure that every future developer builds DragonHub consistently.

21.2 Philosophy

The architecture already solves the difficult problems.

Developers should not redesign architecture.

They should build on it.

Whenever possible

Reuse → Extend → Improve

Never

Rewrite

Architecture

21.3 Before Writing Code

Every feature begins with five questions.

Question 1

Does the Product Specification already define this feature?

If yes → Follow it.

Question 2

Does the Architecture already solve this?

If yes → Reuse it.

Question 3

Does a component already exist?

If yes → Reuse it.

Question 4

Does this duplicate another module?

If yes → Stop.

Question 5

Does this improve basketball workflow?

If no → Do not build it.

21.4 Folder Ownership

Every module owns one folder.

Examples

```
/player-registry  
/club-registry  
/competition-registry  
/statistics  
/fixtures  
/reports  
/tournament-workspace  
/game-notebook  
/player-database
```

Avoid dumping unrelated code into shared folders.

21.5 Component Rules

Never copy components.

Instead

Import

Reuse

Configure

Example

PlayerCard

should exist

once.

Not

PlayerCard2

PlayerCardCompact

PlayerCardLite

PlayerCardMobile

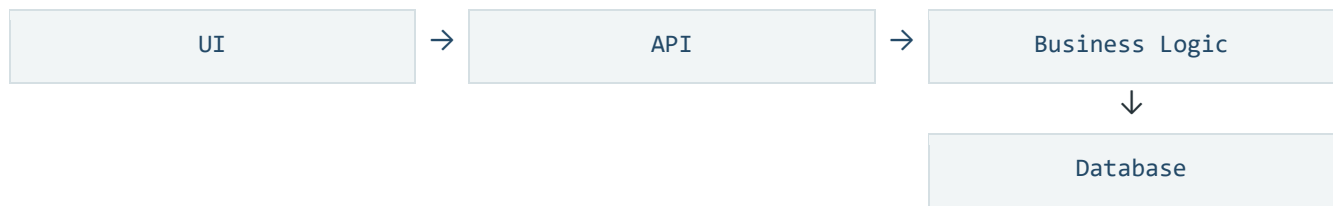
The same component adapts.

21.7 API Rules

UI never contains business logic.

Correct

Figure 21.1 Workflow



Wrong

UI → Database

Business rules belong in services.

Not components.

21.8 State Management

One source of truth.

Never duplicate state.

Example

Player Profile → Player Registry

Not

Player Profile → Own player copy

Everything references the registry.

21.9 Performance Rules

Never optimize prematurely.

Measure first.

Then optimize.

Priority

Queries → Relationships → Rendering → Animations

Animations should never slow basketball workflows.

21.10 Code Review Checklist [Key principle]

Before merging

ask

Does this match Product Specification?

Does this match Architecture?

Does this reuse components?

Does this respect Registries?

Does this improve basketball workflow?

Could another module already solve this?

If any answer is

"No" → [Review again.](#)

21.11 Pull Requests

Every Pull Request should contain

Purpose

Screenshots

Architecture Impact

Modules Changed

QA Checklist

Developer Notes

This creates permanent project history.

21.12 Feature Flags

Large features should ship behind feature flags.

Examples

AI

Transfer Center

Medical

Contract Center

Users should never experience unfinished functionality.

21.13 Testing

Every feature should include

Functional Tests

Workflow Tests

Permission Tests

Performance Tests

Regression Tests

Basketball workflow should always be tested.

Not just code.

21.14 Refactoring

Refactor continuously.

Do not wait

until

"Version 2."

Small improvements every sprint.

Never allow technical debt to accumulate.

21.15 Documentation

No feature is complete until

Product Specification

Architecture Specification

DragonHub Guide

Developer Handbook

are updated.

Documentation ships with code.

21.16 Deprecation

Old features should be

Deprecated → Migrated → Removed

Never leave abandoned functionality indefinitely.

21.17 Code Ownership

Every subsystem has one owner.

Examples

Statistics → Statistics Engine

Registries → Registry Layer

Reports → Knowledge Engine

Avoid multiple implementations.

Rules

21.6 Database Rules

Every query should reference IDs.

Never names.

Correct

PlayerID

Wrong

Player Name

Everything follows Registry architecture.

21.18 Must Never Change

Architecture first.

Code second.

No shortcuts.

No duplicate logic.

No bypassing registries.

No bypassing Event Bus.

Everything remains modular.

Approved Engineering Principles

- Read Product Specification first.
- Read Architecture second.
- Reuse before redesign.
- One owner per subsystem.
- Components never duplicated.
- Registries always used.
- Business logic never belongs in UI.
- Every feature updates documentation.
- Basketball workflow remains the highest priority.

21.19 Success Criteria

A new engineer joining DragonHub should become productive within a few days.

They should not need to reverse engineer the platform.

The documentation should already explain

- why,
- where,
- how,

before they begin coding.

Architecture Notes

I actually think this chapter is what separates an **engineering project** from a **software product**.

Most projects rely on experienced developers remembering unwritten conventions.

DragonHub should not.

Every engineering decision should already exist in the documentation before the first line of code is written.
That means the project becomes **repeatable** rather than **person-dependent**.

Whether Claude, Codex, another AI, or a human engineer joins the project, they should all follow the same engineering rules and produce software that feels like it came from one team.

Related chapters

[Chapter 12 - Component Engine & UI Framework](#)

[Chapter 15 - Feature Development Framework](#)

[Chapter 22 - Database Architecture & Schema Design](#)

[Chapter 23 - API Architecture & Service Layer](#)

[Chapter 24 - Quality Assurance, Testing & Release Management](#)

[Chapter 26 - Engineering Constitution](#)

End of Chapter 21

Chapter 22

Database Architecture & Schema Design

22.1 Purpose

The DragonHub database is designed to preserve basketball knowledge for decades. Unlike traditional sports databases, DragonHub is not optimized only for statistics. It is optimized for relationships, scouting intelligence and long-term knowledge preservation. The database must support:

- professional scouting
- youth scouting
- tournaments
- reports
- recruitment
- historical data
- future AI

without structural redesign.

22.2 Database Philosophy

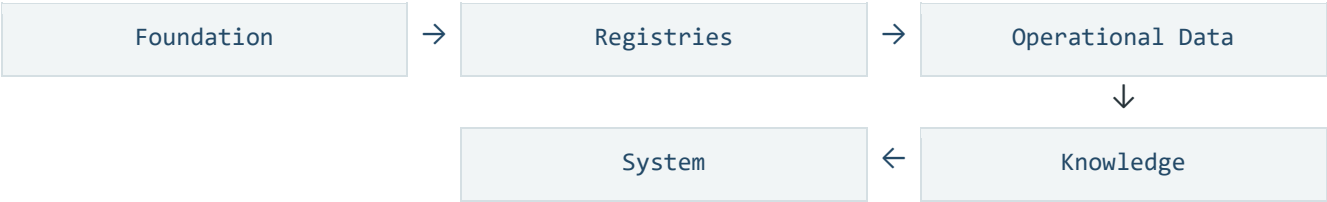
The database follows one simple principle:
Store every piece of information exactly once.
Everything else references it.
Duplicate storage is considered a design error.

Architecture

22.3 Database Layers

The database consists of five logical layers.

Figure 22.1 Workflow



Foundation
Contains

Users

Roles

Permissions

Organizations

Settings

Nothing basketball-specific.

Registries

Contains

Player Registry

Club Registry

Competition Registry

Venue Registry (future)

Agent Registry (future)

Country Registry

Everything receives permanent IDs.

Operational Data

Contains

Games

Fixtures

Standings

Statistics

Rosters

Events

Tournament Workspaces

Game Notebooks

Everything changes regularly.

Knowledge

Contains

Reports

Observations

Scout Summaries

Timeline

Watchlist

Review Queue

Shortlist

Collections

Tags

Everything represents basketball knowledge.

System

Contains

Audit Log

Notifications

Background Jobs

Imports

Search Index

Synchronization

Everything supporting the platform.

22.4 Database Principles

Every table must satisfy

One owner

One responsibility

One primary key

Foreign keys only

No duplicated business logic

22.5 Primary Keys

Every important object uses UUID.

Examples

PlayerID

ClubID

CompetitionID

GameID

ReportID

ObservationID

TimelineID

Never use names as identifiers.

22.6 Foreign Keys

Relationships always use foreign keys.

Example

Observation

```

ObservationID
PlayerID
GameID
CompetitionID
ClubID
AuthorID

```

Never

```

Player Name
Competition Name
Club Name

```

Everything references IDs.

22.7 Naming Standards

Tables

Singular.

Examples

Player

Club

Competition

Report

Observation

Game

Never

Players

Games

Reports

Columns

camelCase

Examples

playerId

clubId

competitionId

createdAt

updatedAt

Never inconsistent naming.

22.8 Soft Deletes [Key principle]

DragonHub almost never permanently deletes records.

Every important table supports

```
deletedAt  
deletedBy  
archived
```

This preserves basketball knowledge.

22.9 Audit Columns

Every important table contains

createdAt

createdBy

updatedAt

updatedBy

This should be automatic.

22.10 Timestamps

All timestamps stored

UTC

Displayed

User timezone

Never store local time.

22.11 Relationships

Use

One-to-One

One-to-Many

Many-to-Many

Never duplicate relationship tables.

Example

```
Players → PlayerReport → Reports
```

Instead of embedding reports.

22.12 Attachments

Media never stored inside database.

Store

```
Reference → Object Storage
```

Supported

Photos

Videos

PDF

Documents

Audio

Everything referenced.

22.13 Search Index

Search never queries every table directly.

Instead

Search Index

references

Players

Clubs

Reports

Competitions

Everything.

Search remains independent.

22.14 Version History

Certain tables support history.

Examples

Scout Summary

Estimated Level

Reports

Notes

Store

[Current Version](#) → [Previous Versions](#)

Never overwrite history.

22.15 Constraints

Examples

Player

must belong to

Player Registry.

Report

must belong to

Player

Club

Competition

or

Game.

Everything validated.

22.16 Indexes

Always index

PlayerID

ClubID

CompetitionID

GameID

EventID

createdAt

updatedAt

Search columns

Never rely on full-table scans.

22.18 Migration Philosophy

Every schema change → Migration → Version → Rollback

Never manually edit production schema.

Rules

22.17 Database Rules

Never store

Derived values.

Examples

Age → Calculate from Birth Date.

Games Played → Calculate from Game table.

Everything computed.

Not duplicated.

22.19 Must Never Change

One UUID per object.

Everything references IDs.

Soft delete by default.

Audit columns everywhere.

History preserved.

Relationships preferred over duplication.

Approved Database Principles

- One source of truth.
- UUID everywhere.
- Foreign keys everywhere.
- Soft deletes by default.
- Audit columns standard.
- Search index separated.
- Media stored externally.
- Derived values calculated.
- Relationships over duplication.
- Historical knowledge preserved.

22.20 Success Criteria

A developer should immediately understand

- where every object lives,
- who owns it,
- what references it,
- how it evolves,
- how it is archived.

The database should remain understandable ten years from now.

Architecture Notes

The database should be treated as the **foundation of DragonHub's institutional memory**.

It is tempting during development to duplicate values for convenience - for example storing player names in reports, competition names in observations, or club names in fixtures. This specification intentionally rejects that approach.

Instead, the database stores **identities and relationships**, while presentation layers assemble the final view.

That decision has two major advantages:

1. **Consistency** - changing a player's name, a club logo, or a competition title updates every module automatically because everything references the same underlying object.
2. **Longevity** - the platform can evolve for decades without accumulating inconsistent historical data.

The database is therefore **not just storage**; it is the canonical representation of the basketball department's knowledge. Every future module should build on this foundation rather than creating its own data structures.

Related chapters

[Chapter 2 - Player Registry](#)

[Chapter 3 - Club Registry](#)

[Chapter 4 - Competition Registry](#)

[Chapter 5 - Relationship Engine](#)

[Chapter 14 - Data Flow & Lifecycle](#)

[Chapter 19 - Security, Permissions & Audit Architecture](#)

[Chapter 23 - API Architecture & Service Layer](#)

End of Chapter 22

Chapter 23

API Architecture & Service Layer

23.1 Purpose

The API Layer is the single gateway between DragonHub clients and the backend.

No frontend component should communicate directly with the database.

Every request follows one architecture.

Client → API → Service → Registry / Engine → Database

This rule has no exceptions.

23.2 Philosophy

Business logic belongs inside services.

Not

Frontend.

Not

Controllers.

Not

Database triggers.

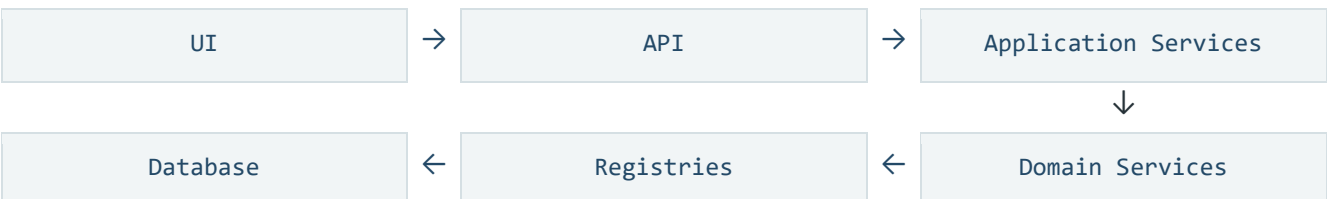
Services own basketball logic.

Architecture

23.3 Architecture

DragonHub follows a layered architecture.

Figure 23.1 Workflow



Each layer has exactly one responsibility.

23.4 API Principles

The API should be

Predictable

Versioned

Stateless

Well documented

Fast

Every endpoint should follow identical conventions.

23.5 API Categories

DragonHub APIs are grouped by domain.

Examples

Players

Clubs

Competitions

Games

Reports

Watchlist

Review Queue

Shortlist

Statistics

Imports

Administration

Every domain owns its endpoints.

23.6 Service Layer [Key principle]

Every module owns one service.

Examples

PlayerService

ClubService

CompetitionService

ReportService

StatisticsService

FixtureService

SearchService

NotificationService

TimelineService

No service should become responsible for another domain.

23.7 Registry Access

Services never bypass registries.

Correct

```
PlayerService → Player Registry
```

Wrong

```
PlayerService → Database
```

Everything flows through the architecture.

23.8 API Versioning

Every public endpoint belongs to a version.

Example

```
/api/v1/player  
/api/v1/report  
/api/v1/club
```

Breaking changes always create a new version.

23.9 Response Format

Every API response follows the same structure.

Example

```
success  
data  
meta  
errors
```

Responses should remain predictable.

23.10 Pagination

Large collections support

Cursor Pagination

Infinite Scroll

Avoid page numbers.

The API should naturally support Infinite Scroll throughout DragonHub.

23.11 Search Endpoints

Global Search becomes its own service.

Search requests should never query individual modules directly.

Instead

```
Search Service → Search Index → Results
```

Everything grouped automatically.

23.12 Authorization

Permissions are checked

inside services.

Never inside UI.

Every endpoint validates

Authentication → Authorization → Object Permissions

before returning data.

23.13 Validation

Every request is validated.

Examples

Player exists.

Club exists.

Competition exists.

Report belongs to player.

Invalid requests never reach business logic.

23.14 Event Publishing

Every successful write operation publishes an event.

Example

Create Report → ReportCreated → Event Bus → Timeline → Search → Notifications

The API does not update other modules directly.

23.15 Error Handling

Every error follows one format.

Examples

Validation Error

Permission Error

Import Error

Registry Error

Unexpected Error

No inconsistent error responses.

23.16 Caching

Frequently requested endpoints support caching.

Examples

Player Profile

Dashboard
Calendar
Club Profile
Competition Profile
Search Suggestions
Everything configurable.

23.17 Documentation

Every endpoint includes
Description
Parameters
Permissions
Responses
Examples
Errors
Documentation is generated automatically whenever possible.

23.18 Future Clients

The same API should support

Web → iPad → Phone → Desktop → Future Mobile Apps → Future AI

No client-specific logic.

Rules

23.19 Must Never Change

Business logic belongs to services.
Registries remain the single source of truth.
Clients never access the database directly.
Every write publishes an event.
Every endpoint follows one response format.

Approved API Principles

- One API layer.
- Services own business logic.
- Registries own identities.
- Clients never access the database directly.
- Every write publishes an event.
- Consistent response format.

- Cursor-based pagination.
- Search is centralized.
- API versioning is mandatory.
- Documentation ships with the API.

23.20 Success Criteria

A new frontend should be able to use DragonHub without requiring backend changes.

Likewise, backend improvements should not require frontend redesigns.

The API becomes a stable contract between every part of the platform.

Architecture Notes

One of the easiest ways for large applications to become difficult to maintain is allowing frontend code to slowly accumulate business logic.

DragonHub should actively avoid that.

A page should never "know" how a player is promoted from the Youth Database to the Pro Database, how club aliases are resolved, or how statistics are refreshed.

Those rules belong inside services.

This creates three major benefits:

1. **Consistency** - every client behaves identically because they all use the same services.
2. **Maintainability** - changing business rules affects one place instead of dozens of screens.
3. **Future-proofing** - future clients (mobile apps, AI assistants, external integrations) automatically inherit the same basketball logic without reimplementing it.

The API Layer therefore becomes much more than a transport mechanism - it becomes the stable contract that keeps every part of DragonHub aligned as the platform grows.

Related chapters

Chapter 5 - Relationship Engine

Chapter 13 - Event Bus & Workflow Engine

Chapter 18 - Integration Layer & External Data Providers

Chapter 19 - Security, Permissions & Audit Architecture

Chapter 22 - Database Architecture & Schema Design

Chapter 24 - Quality Assurance, Testing & Release Management

End of Chapter 23

Chapter 24

Quality Assurance, Testing & Release Management

24.1 Purpose

DragonHub should not be considered complete when code compiles.

A feature is complete only when

- it functions correctly,
- it follows the Product Specification,
- it follows the Architecture,
- it improves basketball workflow,
- and it does not break existing functionality.

Testing therefore includes both technical validation and basketball validation.

24.2 Philosophy

Testing should answer two independent questions.

Technical QA

Does it work?

Basketball QA

Does it improve the workflow?

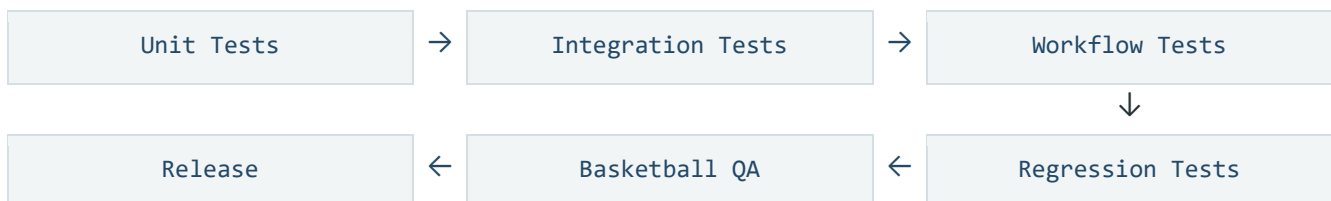
A feature passing Technical QA but failing Basketball QA should not be released.

Architecture

24.3 Testing Pyramid

DragonHub follows a layered testing strategy.

Figure 24.1 Workflow



Every release passes through every layer.

24.4 Unit Testing

Each service should be tested independently.

Examples

Player Registry

Club Registry

Competition Registry

Statistics Engine

Fixture Engine

Search

Relationship Engine

Every service validates its own behaviour.

24.5 Integration Testing

Verify communication between modules.

Examples

Tournament Workspace → Player Registry → Relationship Engine → Timeline

Everything should remain connected.

24.6 Workflow Testing [Key principle]

Workflow testing is unique to DragonHub.

Instead of testing buttons,

test basketball workflows.

Example

Tournament → Import Teams → Roster → Players → Observations → Reports → Timeline →
Watchlist → Review Queue → Shortlist

Everything should work without manual intervention.

24.7 Regression Testing

Every release verifies

Dashboard

Calendar

Events

Tournament Workspace

Game Notebook

Scouting Matchup

Reports

Player Database

Player Profile

Watchlist

Review Queue

Shortlist

Team List

Youth Database

KZS

Nothing should regress.

24.8 UI Validation

Every page should verify

Spacing

Typography

Colors

Buttons

Drawers

Filters

Infinite Scroll

Dark Theme

Light Theme

Nothing should drift from the Design System.

24.9 Data Validation

Every release verifies

Player Registry

Club Registry

Competition Registry

Relationships

Statistics

Fixtures

Profile Photos

Club Logos

No duplicated data.

24.10 Performance Testing

Measure

Dashboard

Player Search

Player Profile

Tournament Workspace

Scouting Matchup

Reports

Everything should remain within performance targets.

24.11 Basketball QA

Examples

Can a scout

watch a game

without interruptions?

Can Tournament Workspace

be used for an entire tournament?

Can a player be found within seconds?

Does Watchlist naturally lead to Review Queue?

Does Calendar still only show club operations?

Basketball workflows always receive final approval.

24.12 Release Checklist

Before every release verify

Product Specification

Architecture Specification

Design System

Developer Handbook

Basketball QA

Performance

Regression

Documentation

Everything complete.

24.13 Feature Freeze

Large features should stabilize before release.

Avoid continuous redesign during implementation.

Feature Freeze → QA → Release → Feedback → Iteration

24.14 Bug Classification

Critical → Basketball workflow broken.

Major → Feature unusable.
Minor → UI inconsistency.
Cosmetic → Visual polish only.

Prioritize by basketball impact.

24.15 User Acceptance Testing

Whenever possible,

test with actual basketball staff.

Professional Scout

Youth Scout

Coach

Front Office

Administrator

Real basketball workflows should validate every release.

24.16 Performance Benchmarks

Examples

Dashboard

< 1 second

Player Search

Instant

Player Profile

< 1 second

Tournament Workspace

< 2 seconds

Scouting Matchup

Immediate

Anything slower requires investigation.

24.17 Documentation Validation

Every released feature updates

DragonHub Guide

Product Specification

Architecture Specification

Developer Handbook

Documentation is part of the release.

Rules

24.18 Must Never Change

No release bypasses Basketball QA.

No feature skips Regression Testing.

Documentation always updates.

Basketball workflows remain more important than technical elegance.

Approved QA Principles

- Basketball QA is mandatory.
- Workflow testing is as important as unit testing.
- Every release includes regression testing.
- Performance targets remain measurable.
- Documentation ships with every release.
- Feature Freeze precedes release.
- Bug priority is determined by basketball impact.
- Real basketball users validate workflows.

24.19 Success Criteria

Every DragonHub release should improve

Reliability

Performance

Consistency

Basketball workflow

without introducing regressions.

Users should trust that updates make the platform better rather than different.

Architecture Notes

One of the biggest lessons from the recent development cycle was that **technical success does not necessarily equal product success**.

The Scouting Matchup redesign is the perfect example:

Technically, the page worked.

Product-wise, it moved away from the workflow that made it valuable.

This chapter exists to prevent that.

Future releases should always be evaluated through two lenses:

1. Can the software perform the task?

2. Does the basketball department actually want to perform the task this way?

Only when both answers are **yes** should the feature be considered complete.

Related chapters

Chapter 12 - Component Engine & UI Framework

Chapter 15 - Feature Development Framework

Chapter 20 - Scalability, Infrastructure & Deployment

Chapter 21 - Engineering Playbook

Chapter 25 - Decision Framework & Product Governance

End of Chapter 24

Part V - Engineering Governance

Chapter 25 Decision Framework & Product Governance

Chapter 26 Engineering Constitution

[Architecture Diagram Placeholder]

Chapter 25

Decision Framework & Product Governance

25.1 Purpose

DragonHub will continue evolving for many years.
Hundreds of design decisions will be made.
New developers will join.
New AI tools will participate.
Without clear governance, the platform will gradually lose consistency.
This chapter defines **how product decisions are made**.

25.2 Philosophy

Every decision should support

Basketball → Workflow → Architecture → Technology

This order must never change.
Technology should never drive product direction.
Basketball should.

Architecture

25.3 Decision Hierarchy

Every proposed feature follows the same hierarchy.

Figure 25.1 Workflow



No feature starts from engineering.

25.4 Decision Authority

Different decisions belong to different documents.

Product Specification

Owns

Purpose

Workflow

UX

Modules

User Experience

Architecture Specification

Owns

Data

Registries

Services

Integrations

Infrastructure

Design System

Owns

Spacing

Components

Colors

Typography

Responsive Layout

Interaction

Developer Handbook

Owns

Coding Standards

Folder Structure

Testing

Deployment

No document should redefine another.

25.5 Conflict Resolution

Whenever two documents disagree

Priority

Product Specification → Architecture → Design System → Developer Handbook

The Product Specification always wins.

Because product intent defines implementation.

25.6 Feature Approval

Every significant feature answers

Why?

Which basketball problem does it solve?

Where?

Which module owns it?

Who?

Which users benefit?

Reuse?

Can existing architecture support it?

Impact?

Which modules are affected?

Documentation?

Which books must be updated?

Only after these questions are answered should development begin.

25.7 Redesign Rules [Key principle]

DragonHub discourages redesigns.

Instead

Improve.

Refactor.

Simplify.

Reuse.

A redesign requires justification.

Example

Good

Current workflow requires six clicks. → Redesign.

Bad

"This looks more modern." → Do not redesign.

25.8 Architecture Protection

Certain architectural decisions are protected.

Examples

Player Registry

Club Registry

Competition Registry

Relationship Engine

Scouting Matchup Layout

Calendar vs Events

Tournament Workspace

These require explicit approval before modification.

25.9 Technical Debt

Technical debt should never accumulate unnoticed.

Every quarter

Review

Duplicate Logic

Duplicate UI

Unused Components

Slow Queries

Architecture Drift

Refactor continuously.

25.10 Product Drift

Every six months

Compare the application

against

Product Specification.

Questions

Does this module still serve its purpose?

Has another module duplicated it?

Has complexity increased?

Does the workflow remain basketball-first?

25.11 Design Drift

Compare every module against the Design System.

Examples

Spacing

Buttons

Cards

Drawers

Search

Filters

Typography

Everything should remain visually consistent.

25.12 Documentation Governance

Every significant feature updates

Product Specification

Architecture Specification

Design System

Developer Handbook

DragonHub Guide

No documentation debt.

25.13 AI Governance

Claude

Responsible for

Frontend

Layout

Interaction

Components

Codex

Responsible for

Backend

Performance

Architecture

Infrastructure

Product Specification

Always overrides implementation suggestions.

25.14 Product Review Meetings

Before major releases

Review

Architecture

Workflow

Design

Performance

Documentation

Every release should improve DragonHub rather than simply adding features.

Rules

Approved Governance Principles

- Basketball drives product decisions.
- Product Specification is the highest authority.
- Architecture implements the product.
- Design System defines visual language.
- Documentation evolves together with code.
- Redesigns require justification.
- Technical debt is actively managed.
- Product drift is reviewed regularly.
- AI follows the specification, not personal preference.

25.15 Success Criteria

DragonHub should remain

Consistent

Understandable

Maintainable

for

10+

years

without losing its identity.

Architecture Notes

This chapter may seem unusual for a software architecture document, but I actually think it's one of the most valuable.

The biggest risk to DragonHub is **not bad code**.

It's **losing its identity**.

Over the last few months we've already seen examples:

- the Scouting Matchup drifted away from live scouting,
- Tournament Workspace briefly became a document editor,
- UI components started diverging,
- workflows became less obvious.

None of those problems were caused by poor engineering - they were caused by a lack of shared product governance.

This chapter exists to prevent that.

It ensures that, years from now, DragonHub will still feel like **the same product**, even after thousands of commits and many different developers have contributed to it.

Related chapters

[Chapter 15 - Feature Development Framework](#)

[Chapter 21 - Engineering Playbook](#)

[Chapter 24 - Quality Assurance, Testing & Release Management](#)

[Chapter 26 - Engineering Constitution](#)

End of Chapter 25

Chapter 26

Engineering Constitution

26.1 Purpose

The Engineering Constitution defines the permanent engineering principles of DragonHub.

Unlike coding standards or implementation details, these principles are considered permanent.

Every architectural decision should be evaluated against them.

If an implementation conflicts with this chapter, the implementation - not the Constitution - should be reconsidered.

26.2 Philosophy

DragonHub is engineered to support basketball operations.

Engineering exists to serve basketball workflows.

Never the opposite.

Whenever engineering and basketball requirements conflict, the architecture should first attempt to satisfy the basketball workflow while maintaining long-term maintainability.

Architecture

26.3 Principle 1 - Product Before Code

Every implementation begins with the Product Specification.

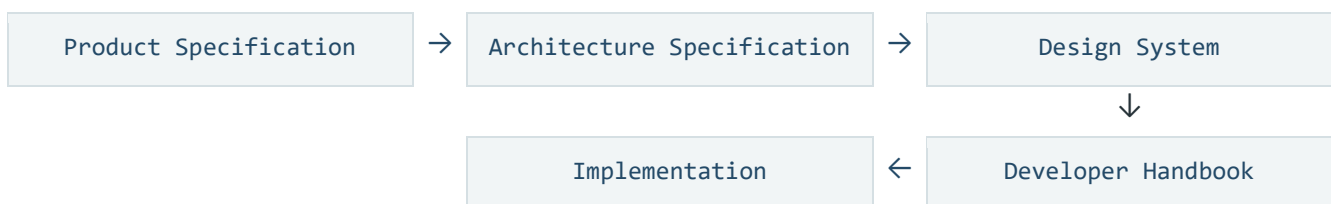
Architecture implements the Product.

Code implements the Architecture.

No implementation should define product behaviour independently.

Hierarchy

Figure 26.1 Workflow



26.4 Principle 2 - One Source of Truth

Every important basketball object exists exactly once.

Player → Player Registry

Club → Club Registry

Competition → Competition Registry

Everything else references those identities.

Duplicate ownership is prohibited.

26.5 Principle 3 - Relationships Over Duplication

DragonHub stores relationships.

Not copies.

Whenever information already exists elsewhere, create a relationship.

Never duplicate the data.

Rules

26.6 Principle 4 - Basketball Workflow First

Every engineering decision should answer

Does this improve the basketball workflow?

If the answer is no,

the feature should be reconsidered.

26.7 Principle 5 - Reuse Before Creating

Before creating

- a new service,
- a new component,
- a new table,
- a new workflow,

ask:

Can an existing subsystem solve this?

Reuse always takes priority.

26.8 Principle 6 - Progressive Loading

Users should begin working immediately.

Load order

Layout → Identity → Interaction → Media → Statistics → Secondary Information

Rendering should never wait for non-essential data.

26.9 Principle 7 - Automation Without Losing Control

Automation removes administration.

Automation never replaces basketball judgment.

DragonHub may suggest.

People decide.

26.10 Principle 8 - Knowledge Is Permanent

Statistics can be imported again.

Fixtures can be imported again.

Reports,

Scout Summaries,

Observations,

Recruitment Decisions,

cannot.

Scouting knowledge is DragonHub's most valuable asset.

26.11 Principle 9 - Every Module Owns One Responsibility

Modules should never compete.

```
Dashboard → Operations
Calendar → Club Planning
Events → Scouting Opportunities
Tournament Workspace → Tournament Scouting
Game Notebook → Live Observation
Reports → Knowledge
Watchlist → Monitoring
Review Queue → Scouting Tasks
Shortlist → Recruitment
```

Responsibilities should remain stable.

26.12 Principle 10 - Evolution Over Reinvention

DragonHub should evolve continuously.

It should almost never require complete redesigns.

Improve.

Refactor.

Extend.

Protect existing workflows.

26.13 Principle 11 - Architecture Is Long-Term

Architecture decisions should be evaluated over

5-10 years.

Not the next sprint.

Every subsystem should support future growth without requiring structural redesign.

26.14 Principle 12 - Documentation Is Part of the Product

Every feature updates

Product Specification

Architecture Specification

Design System

Developer Handbook

DragonHub Guide

Code without documentation is incomplete.

26.15 Principle 13 - Consistency Is a Feature

Consistency improves productivity.

Users should never need to relearn DragonHub.

Every module should feel familiar.

Consistency is considered part of the architecture.

26.16 Principle 14 - DragonHub Is a Basketball Operating System

DragonHub is not

- a statistics website,
- a scouting database,
- a report manager.

DragonHub integrates

Planning

Scouting

Knowledge

Recruitment

into one connected platform.

Every future feature should strengthen that vision.

Engineering Constitution

- Product drives Architecture.
- Architecture drives Implementation.
- One source of truth.
- Relationships over duplication.
- Basketball workflow before technical convenience.
- Reuse before creating.
- Progressive loading everywhere.

- Automation supports people.
- Knowledge is permanent.
- One responsibility per module.
- Evolution over reinvention.
- Documentation is mandatory.
- Consistency is part of the architecture.
- DragonHub remains a Basketball Operating System.

Success Criteria

Related acceptance criteria: Chapter 25, section 25.15.

Architecture Notes

26.17 Final Engineering Statement

The architecture exists to preserve and organize basketball knowledge across many seasons, many scouts and many generations of players.

Technology changes.

Data providers change.

Developers change.

The basketball knowledge of the organization should remain intact.

Every architectural decision should ultimately support that goal.

Related chapters

Chapter 1 - System Architecture & Core Principles

Chapter 15 - Feature Development Framework

Chapter 19 - Security, Permissions & Audit Architecture

Chapter 21 - Engineering Playbook

Chapter 25 - Decision Framework & Product Governance

End of Chapter 26

Glossary

Alias

An alternative or historical name that resolves to the same registry identity.

See Chapter 2, Chapter 3, Chapter 4, Chapter 10

Architecture ownership

The subsystem responsible for an object or capability; other modules consume it.

See Chapter 1, Chapter 14, Chapter 21

Audit trail

A record of important actions, authors, changes, and prior values.

See Chapter 14, Chapter 19, Chapter 22

Background job

An independent task that synchronizes or processes data without blocking the user.

See Chapter 8, Chapter 20

Club Registry

The shared identity, aliases, history, logos, and roster relationships for basketball organizations.

See Chapter 3

Competition Registry

The shared competition identity, season context, sources, and relationships.

See Chapter 4

Connector

A provider-specific integration that supplies normalized data through the Integration Layer.

See Chapter 18

Derived value

A value calculated from existing data rather than maintained as a separate source of truth.

See Chapter 22

Estimated Level

A basketball projection that remains under human control, distinct from following a player.

See Chapter 9

Event Bus

The shared mechanism through which published events trigger independent consumers.

See Chapter 13

Fixture Engine

The shared scheduling system that distributes competition fixtures to relevant workflows.

See Chapter 7

Global Search Engine

The central, alias-aware and context-aware search capability consumed by every module.

See Chapter 10

Intelligence Engine

The system that surfaces signals, suggestions, and changes while leaving basketball decisions to people.

See Chapter 9

Knowledge Management Engine

The system that preserves observations, reports, opinions, and decisions over time.

See Chapter 16

Manual override

An explicit human correction that subsequent imports must respect.

See Chapter 8, Chapter 14, Chapter 18, Chapter 19

Player Registry

The permanent player identity and linked basketball knowledge referenced by all modules.

See Chapter 2

Progressive loading

Rendering essential identity and interactions before media, statistics, and secondary information.

See Chapter 1, Chapter 12, Chapter 26

Promotion

Moving a player between youth and professional workflows while retaining PlayerID and history.

See Chapter 2, Chapter 14, Chapter 17

Relationship Engine

The shared connections between identified objects, supporting navigation, timelines, and knowledge reuse.

See Chapter 5

Scout Summary

The current scouting opinion; reports preserve the historical opinions that support it.

See Chapter 2, Chapter 16

Service Layer

The owner of business logic exposed to clients through the API.

See Chapter 23

Soft delete

Archiving a record with deletion metadata while preserving historical relationships.

See Chapter 14, Chapter 22

Statistics Engine

The owner of validated statistics, source metadata, freshness, and competition-based refresh policies.

See Chapter 6

Synchronization Engine

The background scheduler and jobs that keep imported information current.

See Chapter 8

Timeline

A chronological view of referenced events and knowledge associated with an object.

See Chapter 2, Chapter 5, Chapter 16

Workflow Automation Engine

The system that prepares and connects workflow steps without making basketball decisions.

See Chapter 17

Abbreviations

Abbreviation	Meaning
AI	Artificial intelligence
API	Application programming interface
CSV	Comma-separated values
FAQ	Frequently asked questions
ID	Identifier
JSON	JavaScript Object Notation
QA	Quality assurance
SSO	Single sign-on
UI	User interface
UTC	Coordinated Universal Time
UUID	Universally unique identifier
UX	User experience

Basketball terms used in examples

Term	Meaning in this specification
ABA, ACB, BBL, LNB	Competition labels used in the statistics refresh policy.
BCL	Basketball Champions League.
BC / BCV	Basketball Club / BC Vienna alias example.
BKT / ULEB	Competition-name variants used in historical naming examples.
EYBL / ANGT	Youth competition labels used in competition examples.
FIBA	International basketball federation referenced as a public data source.
KZS	Slovenian basketball federation referenced as a public data source.
NBA / NCAA	Basketball competition or college-basketball labels used in the source examples.
U16 / U18	Under-16 / under-18 age categories.

Index

A

Alias 12, 20, 27, 71

API architecture 165

Architecture ownership 5, 99, 149

Audit trail 99, 133, 156

Authentication 134

Authorization 134, 168

B

Background job 57, 140

Backups 137, 145

C

Caching 47, 142, 168

Club Registry 20

Code reviews 111, 151

Competition Registry 27

Component Engine 84

Confidential data 136

Connector 127

D

Data lifecycle 99

Database architecture 156

Deployment 144

Derived value 156

Duplicate detection 15, 67

E

Engineering Constitution 186

Engineering Playbook 149

Estimated Level 64

Event Bus 92

F

Feature development 106

Feature flags 152

Fixture Engine 51

Fixture status 54

Foreign keys 158

G

Global Search Engine 71

Governance 179

I

Indexing 162

Integration Layer 127

Intelligence Engine 64

K

Knowledge Management Engine 113

Knowledge ownership 116

M

Manual override 57, 99, 127, 133

Media storage 142, 160

Migration 162

Multi-tenancy 143

N

Notification Engine 78

P

Pagination 167

Permissions 133

Player Registry 12

Profile photos 16, 59

Progressive loading 5, 84, 186

Promotion 12, 99, 120

Provider failover 130

Q

Quality assurance 171

R

Refresh policy 44, 54

Regression testing 172

Relationship Engine 35

Release management 171

Roster ownership 23

S

Scout Summary 12, 113

Security 133

Service Layer 165

Soft delete 99, 156

Statistics Engine 43

Synchronization Engine 57

T

Timeline 12, 35, 113

V

Versioning 102, 143, 167

W

Workflow Automation Engine 120